

IMPLEMENTATION OF GATEWAY DESIGNING USING NETSIM AND RASPBERRY PI

*Project work submitted in partial fulfillment of the
requirements for the award of degree of*

Bachelor of Technology in Electronics and Communication Engineering

VANAPALLI JYOTHSNA (322103312216)

Y. BHANU CHANDRA (322103312226)

T. DILIP KUMAR RAJU (322103312206)

TATISETTY NAGENDRA(322103312207)

Under the guidance of

Name of the supervisor

Dr. U. N. V. P. RAJENDRANATH

Assistant Professor



**COLLEGE OF ENGINEERING
(AUTONOMOUS)**

**Department of Electronics and Communication Engineering
GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING (AUTONOMOUS)
(Affiliated to Andhra University, Visakhapatnam, A.P)
VISAKHAPATNAM- 530048**

April, 2025-26



GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING (Autonomous)

Approved by AICTE & Affiliated to Andhra University, Visakhapatnam from 2022-23
(Affiliated to JNTUK, Kakinada upto 2021-22)

Accredited by NAAC at 'A++' Grade for 7 years in the 3rd Cycle



ELECTRONICS AND COMMUNICATION ENGINEERING DEPARTMENT

CERTIFICATE

This is to certify that the project titled “ **IMPLEMENTATION OF GATEWAY DESIGNING USING NETSIM AND RASPBERRY PI** ” is a Bonafide record of the work done by **V.JYOTHSNA (322103312216), Y.BHANU CHANDRA (322103312226), T.DILIP KUMAR RAJU (322103312206), T.NAGENDRA (322103312207)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Electronics and Communication Engineering** of the Gayatri Vidya Parishad College of Engineering (Autonomous) affiliated to Andhra University, Visakhapatnam during the year **2025-2026**.

Under the guidance of:

Head of the Department:

Dr. U. N. V. P. Rajendranath
Assistant Professor,
Dept. of ECE,
GVPCE (A)

Dr. N. Deepika Rani
Professor & HoD,
Dept. of ECE,
GVPCE (A)

Project Viva-voce held on _____

Signature of the Internal Examiner

Signature of the External Examiner

DECLARATION

This is to certify that the project titled “**IMPLEMENTATION OF GATEWAY DESIGNING USING NETSIM AND RASPBERRY PI**” is a Bonafide record of the work done by **V.JYOTHSNA (322103312216), Y.BHANU CHANDRA (322103312226), T.DILIP KUMAR RAJU (322103312206), T. NAGENDRA (322103312207)** in partial fulfillment of the requirement for the award of degree of B. Tech. in Electronics and Communication Engineering to Gayatri Vidya Parishad College of Engineering (Autonomous), affiliated to Andhra University, Visakhapatnam comprises only our original work and due acknowledgement has been made in the text to all other material used.

Date:

V. JYOTHSNA (322103312216)

Y. BHANU CHANDRA (3221013312226)

T. DILIP KUMAR RAJU (322103312206)

T. NAGENDRA (322103312207)

ACKNOWLEDGEMENT

We take this opportunity to thank one and all who have helped in making this possible. We are grateful to Gayatri Vidya Parishad College of Engineering (Autonomous), for giving the opportunity to work on a project report as a part of the curriculum.

Our sincere thanks to our guide **Dr. U. N. V. P. Rajendranath**, Assistant Professor, Academics (UG), Electronics and Communication Engineering Department, for his continuous and valuable guidance.

With a great sense of pleasure and privilege, we extend our gratitude and sincere thanks to, **Dr. N. Deepika Rani**, Professor and Head of the Electronics and Communication Engineering Department, for her encouragement.

With a great sense of pleasure and privilege, we extend our gratitude and sincere thanks to **Dr. A. Bala Koteswara Rao**, Principal, Gayatri Vidya Parishad College of Engineering (Autonomous) for his continuous encouragement during the course of study.

We would like to express our sincere thanks to all the faculty and staff of the Department of Electronics and Communication Engineering for their advice and cooperation.

We owe our thanks much more than our words can express to our families who sacrificed in all respects with love and affection to complete this thesis work satisfactorily.

Finally, we take this opportunity to thank all the people who helped us in completion of thesis work, directly or indirectly, and for their timely encouragement and faithful services.

ABSTRACT

This project presents the design and implementation of an IoT-based Wireless Sensor Network (WSN) with an efficient gateway system using both simulation and real-time hardware integration. The system consists of multiple sensor nodes, a gateway, a router, and a cloud platform to enable continuous data transmission from sensors to the cloud.

In the hardware implementation, multiple sensors (DHT11 sensors for temperature and humidity and an MQ3 sensor for gas detection) are interfaced with an Arduino microcontroller, which acts as a data acquisition unit. The collected sensor data is transmitted to a Raspberry Pi, which functions as an IoT gateway. The Raspberry Pi processes the data and uploads it to cloud platforms such as Google Sheets and a web interface developed using GitHub, enabling real-time monitoring and visualization.

In addition to hardware implementation, the system is modeled and simulated using NetSim to analyze network performance under controlled conditions. Two routing protocols, AODV (Ad hoc On-Demand Distance Vector) and RPL (Routing Protocol for Low-Power and Lossy Networks), are implemented and compared using Constant Bit Rate (CBR) traffic.

Performance evaluation is carried out using key metrics such as throughput, end-to-end delay, packet delivery ratio (PDR), and energy efficiency. To further enhance system efficiency, an edge-based gateway approach is incorporated, where the gateway performs local data processing and intelligent task handling, reducing latency, bandwidth usage, and energy consumption in the network.

The results demonstrate that routing protocol selection and efficient gateway design play a crucial role in improving IoT network performance. RPL provides better energy efficiency and stability for IoT environments, while AODV offers flexibility in dynamic scenarios. The integration of hardware implementation with simulation validates the practicality and effectiveness of the proposed system for real-time IoT applications

CONTENTS

	Page No.
CERTIFICATE	II
DECLARATION.....	III
ACKNOWLEDGEMENT.....	IV
ABSTRACT.....	V
CONTENTS.....	VI
LIST OF ABBREVIATIONS.....	X
LIST OF FIGURES.....	XII
LIST OF TABLES.....	XIII
CHAPTER 1	
INTRODUCTION	
1.1 Internet of Things and Wireless Sensor Network.....	14
1.2 Need for Edge Computing in IoT.....	16
1.3 IoT Gateway and Its Role.....	16
1.4 Problem Statement.....	17
1.5 Proposed Solution.....	17
1.6 Scope of the Project.....	18

1.7 Organization of the Report.....	18
-------------------------------------	----

CHAPTER 2

LITERATURE SURVEY.....	19
-------------------------------	-----------

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Introduction.....	20
3.2 System Architecture.....	20
3.3 Network Configuration and Traffic Model.....	21
3.4 Routing Protocols.....	22
3.5 Scheduling and Queuing Mechanisms.....	23
3.6 Edge-Based TRAPS Mechanism.....	24
3.7 Simulation Scenarios.....	24
3.8 Performance Evaluation Parameters.....	25
3.9 Summary.....	25

CHAPTER 4

IMPLEMENTATION AND SIMULATION SETUP

4.1 Introduction.....	26
4.2 Simulation Tool.....	26
4.3 Network Topology Setup.....	27

4.4 Device Configuration.....	28
4.5 Traffic Configuration.....	29
4.6 Routing Protocol Configuration.....	30
4.7 Scheduling and Queuing Setup.....	30
4.8 Simulation Parameters and Scenarios.....	31
4.9 Performance Metrics Configuration.....	31
4.10 Implementation of Edge-Based Processing.....	31
4.11 Summary.....	32

CHAPTER 5

HARDWARE IMPLEMENTATION

5.1 Introduction.....	33
5.2 Sensors.....	34
5.3 Arduino (Data Acquisition Unit).....	35
5.4 Raspberry Pi 4 (Gateway).....	38
5.5 Communication Protocols Used in Hardware.....	43
5.6 Remote Access using RealVNC Viewer.....	44
5.7 Cloud Storage using Google Sheets.....	45
5.8 Web Application (User Interface).....	48
5.9 Summary.....	51

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Introduction.....	52
6.2 Case 1: Equal Configuration of Sensor Nodes.....	52
6.3 Case 2: Priority and FIFO Scheduling.....	52
6.4 Case 3: Controlled Start Time.....	53
6.5 Throughput Analysis.....	53
6.6 End-to-End Delay Analysis.....	59
6.7 Energy Consumption Analysis.....	59
6.8 Packet Delivery Ratio.....	65
6.9 Impact of Edge-Based TRAPS Mechanism.....	69
6.10 Performance Comparison of AODV and RPL.....	69
6.11 Hardware Results and Analysis.....	71
6.12 Summary.....	75

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion.....	76
7.2 Future Scope.....	77
7.3 Final Remarks.....	77

REFERENCES.....	78
------------------------	-----------

LIST OF ABBREVIATIONS

AODV	: Ad hoc On-Demand Distance Vector
RPL	: Routing Protocol for Low-Power and Lossy Networks
DSR	: Dynamic Source Routing
OLSR	: Optimized Link State Routing
ZRP	: Zone Routing Protocol
MANET	: Mobile Ad hoc Network
WSN	: Wireless Sensor Network
WPAN	: Wireless Personal Area Network
WAN	: Wide Area Network
WIRED	: Wired Network (using cables)
CSV	: Comma-Separated Values
CBR	: Constant Bit Rate
FTP	: File Transfer Protocol
HTTP	: HyperText Transfer Protocol
TCP	: Transmission Control Protocol
OSPF	: Open Shortest Path First
FIFO	: First In First Out
WFQ	: Weighted Fair Queuing
EDF	: Earliest Deadline First
RED	: Random Early Detection
QoS	: Quality of Service

BE : Best Effort

UGS : Unsolicited Grant Service

RTPS : Real-Time Polling Service

ERTPS : Extended Real-Time Polling Service

NRTPS : Non-Real-Time Polling Service

PDR : Packet Delivery Ratio

ADC : Analog to Digital Converter

UART : Universal Asynchronous Receiver Transmitter

USB : Universal Serial Bus

VNC : Virtual Network Computing

API : Application Programming Interface

JSON : JavaScript Object Notation

EMS : Energy Monitoring System

TRAPS : Task Requirements Aware Pre-processing and Scheduling

LIST OF FIGURES

Figure 1.1	Basic IoT Architecture with Wireless Sensor Network.....	12
Figure 1.2	Edge Computing vs Cloud Computing.....	13
Figure 3.1	Proposed IoT System.....	21
Figure 3.2	Constant Bit Rate (CBR) Traffic Model.....	22
Figure 3.3	AODV Routing Protocol.....	23
Figure 3.4	RPL Routing Protocol.....	27
Figure 3.5	Scheduling and Queuing Mechanisms.....	28
Figure 4.1	NetSim IoT Simulation Environment.....	29
Figure 4.2	Uniform Placement of Sensor Nodes.....	29
Figure 4.3	Random Placement of Sensor Nodes.....	30
Figure 4.4	Traffic Configuration using CBR.....	30
Figure 4.5	Sensor Node Configuration.....	30
Figure 5.1	IoT Gateway System Architecture.....	30
Figure 5.2.1	DHT11 Sensors.....	31
Figure 5.2.2	MQ3 Sensors.....	31
Figure 5.3	Arduino.....	33
Figure 5.4	Raspberry Pi 4.....	36
Figure 5.6	Remote Access of Raspberry Pi using RealVNC Viewer.....	41
Figure 5.8	Web Application Output Displaying Live Sensor Data.....	48
Figure 6.11.1	Hardware Setup Output.....	68
Figure 6.11.2(a)	Sensor Data Output (Arduino).....	69
Figure 6.11.2(b)	Sensor Data Output (Raspberry Pi).....	70
Figure 6.11.4	Cloud Storage Results (Google Sheets).....	71
Figure 6.11.5	Web Application Output.....	72

LIST OF TABLES

	Page No
Table 5.2.1: Specifications of Sensors Used in the System.....	32
Table 5.2.2: Arduino Specifications.....	33
Table 5.3: Arduino Pin Configuration.....	33
Table 5.4: Raspberry Pi 4 Specifications.....	36
Table 6.5.1: AODV Application Metrics.....	51
Table 6.5.2: Priority Order (Which Sensor Reach First to Cloud).....	53
Table 6.5.3: RPL Application Metrics.....	54
Table 6.5.4: Priority Order (Which Sensor Reach First to Cloud).....	56
Table 6.7.1: AODV Energy Harvesting (QoS BE – Low).....	57
Table 6.7.2: AODV Energy Efficiency.....	59
Table 6.7.3: RPL Energy Harvesting (QoS BE – Low).....	60
Table 6.7.4: RPL Energy Efficiency.....	62
Table 6.8.1: PDR Table of AODV.....	64
Table 6.8.1: PDR Table of RPL.....	66
Table 6.10.1: Performance Comparison of AODV and RPL.....	67

CHAPTER 1

INTRODUCTION

1.1 Internet of Things and Wireless Sensor Networks

The Internet of Things (IoT) connects physical devices and sensors through the internet to collect and exchange real-time data. It is widely used in areas like environmental monitoring, smart cities, healthcare, and industrial automation.

A key part of IoT is the Wireless Sensor Network (WSN), where sensor nodes collect data such as temperature and humidity and send it to a gateway, which forwards it to the cloud for processing. However, traditional cloud-based IoT systems face challenges like high latency, increased bandwidth usage, network congestion, and energy consumption as the number of devices grows.

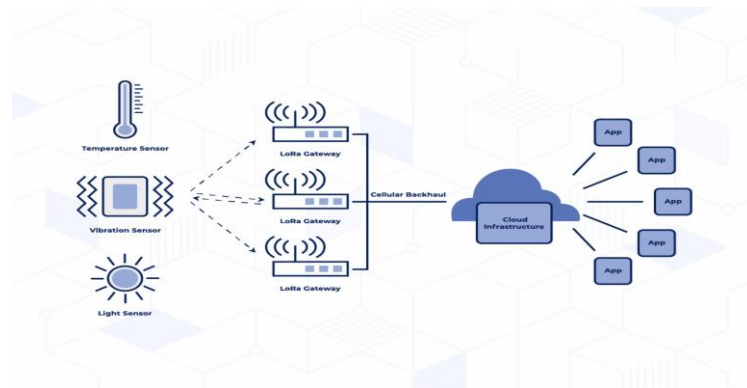


Figure 1.1: Basic IoT Architecture with Wireless Sensor Network

1.2 Need for Edge Computing in IoT

In traditional IoT systems, all sensor data is sent to the cloud for processing, causing delays, high network load, and increased energy consumption.

To overcome these issues, edge computing processes data closer to the source using gateway devices. This reduces latency, bandwidth usage, and improves performance. In this project, an edge-based gateway performs local processing and task management, enabling faster and more efficient IoT operations.

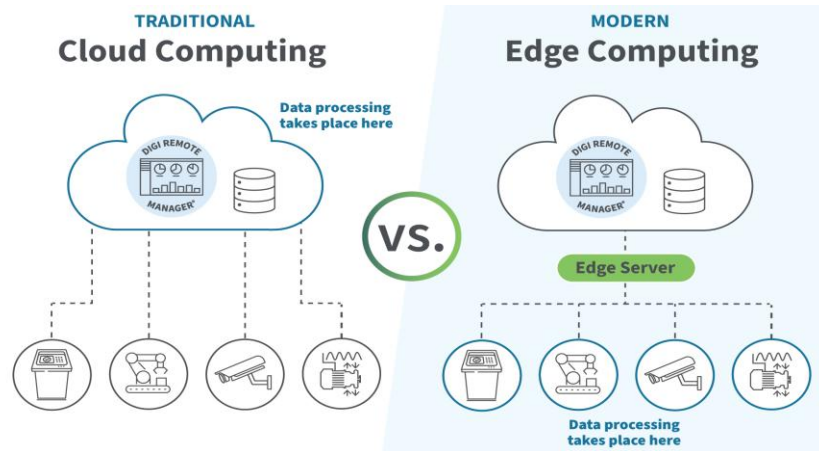


Figure 1.2: Edge Computing vs Cloud Computing

1.3 IoT Gateway and Its Role

The IoT gateway connects Wireless Sensor Networks to the internet, acting as a bridge between sensor nodes and the cloud. It handles data collection, communication, and protocol conversion.

Modern gateways also perform data processing, filtering, and decision-making. By integrating edge computing, data is processed locally, and only relevant information is sent to the cloud. In this project, the gateway uses the TRAPS mechanism to improve task execution efficiency.

1.4 Problem Statement

Traditional IoT systems rely heavily on cloud-based processing, where all sensor data is transmitted to remote servers. This results in increased latency, higher bandwidth consumption,

network congestion, and packet loss, especially when the number of sensor nodes increases. Furthermore, multiple sensor tasks with similar requirements are often executed independently, leading to redundant operations and unnecessary energy consumption. This reduces the overall efficiency and lifetime of the Wireless Sensor Network. Therefore, there is a need for an efficient system that can process data closer to the source, reduce redundant task execution, and optimize resource utilization while maintaining Quality of Service (QoS) requirements.

1.5 Proposed Solution

To overcome the limitations of traditional IoT systems, this project proposes an edge-based gateway system that integrates the TRAPS mechanism for efficient task processing and scheduling.

The TRAPS mechanism consists of three main components:

- A task pre-processor that groups similar tasks to reduce redundancy
- An Energy Monitoring System (EMS) that tracks the residual energy of sensor nodes
- A scheduler that assigns tasks based on QoS requirements and energy availability

By performing task processing at the gateway level, the system reduces unnecessary data transmission and improves network efficiency. This approach also ensures balanced energy consumption among sensor nodes, thereby extending the overall network lifetime.

1.6 Scope of the Project

The main objective of this project is to design and evaluate an efficient IoT gateway using edge computing principles. The scope of the project includes:

- Designing a Wireless Sensor Network connected through a gateway
- Implementing an edge-based processing mechanism using TRAPS
- Simulating the system using NetSim for performance evaluation
- Analyzing different scheduling techniques such as FIFO and Priority

- Evaluating performance metrics including throughput, delay, and energy consumption

The system is tested under different scenarios to study network behavior and to identify the most effective approach for improving efficiency.

1.7 Organization of the Report

This report is organized as follows:

- **Chapter 1** presents the introduction to IoT, WSN, and edge computing along with the problem statement and proposed solution.
- **Chapter 2** discusses the literature survey related to IoT gateways and task scheduling mechanisms.
- **Chapter 3** explains the system design, architecture, and working of the proposed model.
- **Chapter 4** presents the simulation setup and tools used for implementation.
- **Chapter 5** discusses the results and performance analysis.
- **Chapter 6** concludes the project and suggests future enhance

CHAPTER 2

LITERATURE SURVEY

Bharti et al. [1] proposed a Task Requirement Aware Pre-processing and Scheduling (TRAPS) mechanism for IoT sensory environments. The system introduces a gateway-based architecture where tasks are pre-processed before execution in Wireless Sensor Networks (WSNs). It focuses on reducing redundant task execution by grouping tasks with similar Quality of Service (QoS) and functional requirements. The architecture includes a task pre-processor, an Energy Monitoring System (EMS), and a scheduler. The EMS predicts the residual energy of sensor nodes without additional communication overhead, improving energy efficiency. The scheduler assigns tasks based on sensor capability and energy availability. The proposed approach significantly enhances network performance, reduces energy consumption, and minimizes unnecessary data transmission. However, the system increases computational complexity at the gateway level and requires efficient task classification mechanisms for large-scale IoT environments.

Akkaya et al. [2] presented a performance evaluation of routing protocols in Wireless Sensor Networks, focusing on key metrics such as delay, throughput, scalability, and energy consumption. The study compares hierarchical routing protocols including LEACH, PEGASIS, and VGA using simulation models. The results indicate that PEGASIS achieves better network lifetime and energy efficiency compared to other protocols, while LEACH performs moderately well under certain network conditions. VGA, although efficient in data aggregation, shows limitations in energy usage under constrained environments. The paper emphasizes the importance of selecting appropriate routing protocols for efficient IoT communication. However, the analysis is limited to specific simulation scenarios and may not fully represent real-time dynamic IoT environments.

Glória et al. [3] proposed the design and implementation of an IoT gateway for creating smart environments using Raspberry Pi and Arduino. The system enables real-time monitoring and remote control of devices through bidirectional communication between sensor networks and cloud platforms. The gateway supports multiple communication protocols and integrates hardware and software components, including a web platform and MQTT communication. It

allows users to monitor environmental parameters such as temperature, humidity, and light, and control devices remotely. The system is cost-effective, flexible, and scalable for various IoT applications. However, the system relies on continuous connectivity and may face limitations in handling large-scale deployments and high data loads.

Shepherd et al. [4] proposed EmLog, a tamper-resistant logging system for constrained embedded devices using Trusted Execution Environments (TEEs). The system ensures secure and reliable logging by protecting log data against unauthorized modification, even in the presence of software-based attacks. It utilizes ARM TrustZone and GlobalPlatform TEE to isolate logging operations from the main operating system. The approach incorporates cryptographic techniques such as hash chaining and HMAC to maintain data integrity. Additionally, the system supports secure storage and remote verification, enabling effective forensic analysis after system failures. Performance evaluation shows low memory overhead and high throughput. However, the dependency on TEE-based architectures limits its applicability in low-cost embedded systems.

Sarojini et al. [5] developed a smart real-time data logging system for industrial automation using an I2C-based communication protocol. The system integrates sensors, an Arduino microcontroller, RTC module, and SD card to continuously monitor machine parameters such as temperature and operational status. It provides real-time data acquisition along with timestamping for future analysis and supports remote monitoring through web-based interfaces. The use of I2C protocol enables efficient multi-device communication with minimal wiring, making the system cost-effective and scalable. However, the system generates large volumes of unstructured data and lacks advanced data validation and event-based logging mechanisms, which makes failure analysis more complex.

Zanella et al. [6] presented a comprehensive survey on Internet of Things (IoT) applications, communication technologies, and challenges. The study discusses various IoT domains such as smart cities, healthcare, and industrial automation, highlighting the role of communication protocols like ZigBee, Bluetooth, and Wi-Fi. It emphasizes the need for interoperability, scalability, and efficient data management in IoT systems. The paper also identifies key challenges including security, energy efficiency, and standardization. Although the survey provides a broad overview, it does not focus on specific implementation strategies for real-time

systems.

Al-Fuqaha et al. [7] reviewed IoT architectures, enabling technologies, and communication protocols with a focus on integration with cloud computing and big data analytics. The study highlights the importance of middleware, data processing, and service management in IoT ecosystems. It discusses various challenges such as heterogeneity, scalability, and security issues in IoT deployments. The paper provides insights into future research directions, including edge computing and intelligent data processing. However, it mainly focuses on theoretical aspects and lacks practical implementation details.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Introduction

This chapter presents the design and methodology of the IoT-based WSN system with an edge computing gateway. The system aims to improve efficiency, reduce latency, and optimize energy consumption through intelligent processing at the gateway.

It combines edge computing with advanced task scheduling and is implemented using NetSim, focusing on efficient data transmission, routing, and resource utilization.

3.2 System Architecture

The system architecture includes sensor nodes, a gateway, a router, and a cloud server. Sensor nodes collect environmental data and transmit it wirelessly to the gateway, which connects the network to the internet. The gateway sends data to the cloud through a router using wired communication. In simulation, sensor nodes are deployed using uniform placement (grid pattern) or random placement to study network behavior.

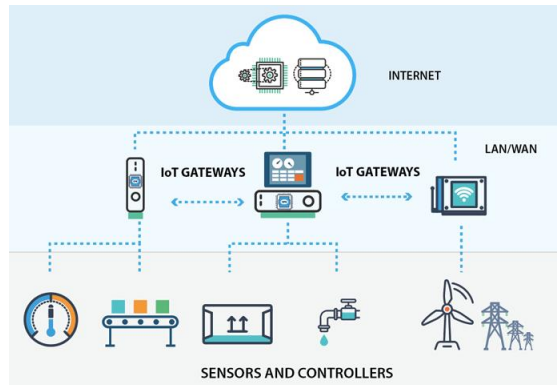


Figure 3.1: Proposed IoT System Architecture

3.3 Network Configuration and Traffic Model

The network is configured in NetSim using the IoT module, where sensor nodes generate traffic based on a Constant Bit Rate (CBR) model for continuous data transmission. Each node is set with parameters like packet size, transmission interval, and start/stop time to analyze different conditions.

Quality of Service (QoS) is also applied, classifying traffic into categories such as Best Effort (BE), Real-Time Polling Service (RTPS), and Unsolicited Grant Service (UGS) with different priority levels.

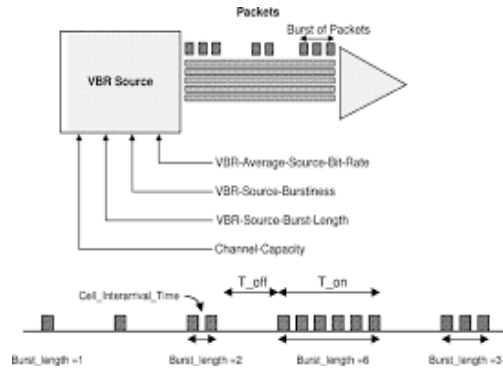


Figure 3.2: Constant Bit Rate (CBR) Traffic Model

3.4 Routing Protocols

The system supports two routing protocols at the network layer: AODV (Ad hoc On-Demand Distance Vector) and RPL (Routing Protocol for Low-Power and Lossy Networks). AODV is a reactive routing protocol that establishes routes only when required, thereby reducing routing overhead. It is suitable for dynamic networks where topology changes frequently. On the other hand, RPL is specifically designed for IoT environments and supports low-power and lossy networks by constructing a hierarchical routing structure. The performance of both protocols is analyzed based on metrics such as energy efficiency, delay, and packet delivery ratio. This

comparison helps in identifying the most suitable routing protocol for IoT-based sensor networks.

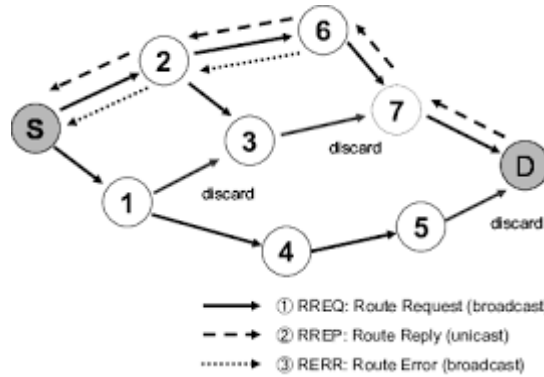


Figure 3.3: AODV (Ad hoc On-Demand Distance Vector) routing protocol

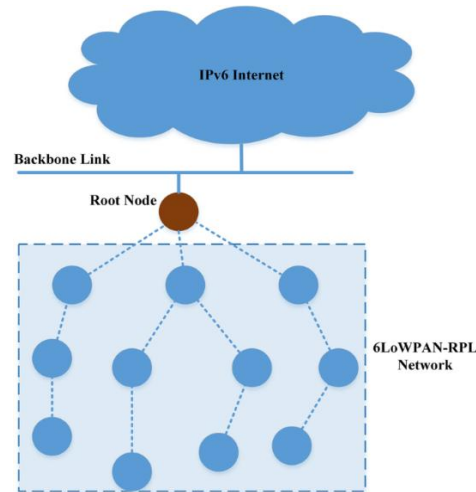


Figure 3.4: RPL (Routing Protocol for Low-Power and Lossy Networks) routing protocol

3.5 Scheduling and Queuing Mechanisms

The gateway supports multiple scheduling techniques, including FIFO (First-In-First-Out), Priority Scheduling, Round Robin, Weighted Fair Queuing (WFQ), and Earliest Deadline First (EDF). FIFO scheduling processes packets in the order of arrival, while priority scheduling assigns precedence to high-priority packets. WFQ distributes bandwidth based on assigned

weights, and EDF is used for time-critical applications. In addition to scheduling, queuing mechanisms such as Drop Tail, Random Early Detection (RED), and Weighted RED (WRED) are used to manage packet transmission and avoid congestion. These mechanisms play a crucial role in controlling network performance and packet loss.

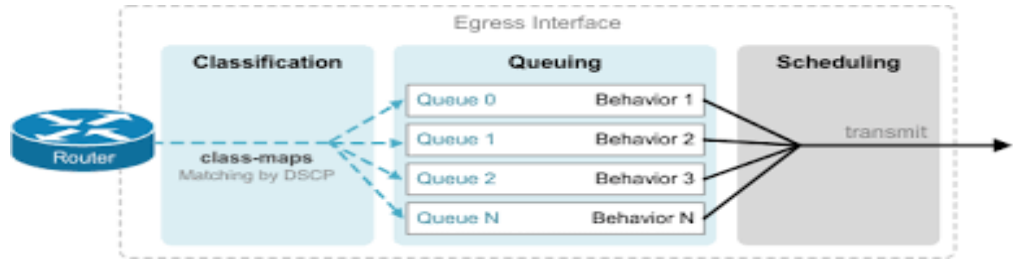


Figure 3.5: Scheduling and Queuing Mechanisms

3.6 Edge-Based TRAPS Mechanism

To enhance network efficiency, the proposed system integrates the TRAPS (Task Requirements Aware Pre-processing and Scheduling) mechanism at the gateway level. The TRAPS mechanism performs intelligent task processing by grouping similar tasks and eliminating redundant executions. It consists of three main components: task pre-processing, energy monitoring, and scheduling. The task pre-processor reduces duplication by combining similar tasks, while the Energy Monitoring System tracks the residual energy of sensor nodes. The scheduler then assigns tasks based on QoS requirements and available energy, ensuring efficient resource utilization and prolonged network lifetime.

3.7 Simulation Scenarios

The system is evaluated under multiple simulation scenarios to analyze network behavior:

- **Case 1: Equal Configuration**

All sensor nodes are configured with identical parameters such as packet size,

transmission interval, and timing. The results show that packet arrival at the cloud is random due to wireless channel contention and medium access delays.

- **Case 2: Priority and FIFO Scheduling**

Priority levels are assigned to sensor nodes, and FIFO scheduling is applied. However, due to the non-deterministic nature of wireless communication, packet delivery order remains unpredictable.

- **Case 3: Controlled Start Time**

The start times of sensor nodes are varied while keeping other parameters constant. This results in controlled and predictable packet delivery, demonstrating that transmission timing plays a significant role in network behavior.

3.8 Performance Evaluation Parameters

The performance of the system is evaluated using key metrics such as throughput, end-to-end delay, energy efficiency, and packet delivery ratio. Energy efficiency is calculated based on the ratio of remaining energy to initial energy of sensor nodes. The simulation results indicate that AODV achieves slightly higher energy efficiency compared to RPL under certain conditions, while RPL provides better adaptability for IoT-specific environments. The Packet Delivery Ratio (PDR) is also analyzed, which measures the reliability of data transmission. The results show variations in PDR across different sensor nodes due to network conditions and scheduling behavior.

3.9 Summary

This chapter presented the detailed system design and methodology of the proposed IoT-based edge computing system. The integration of routing protocols, scheduling mechanisms, and the TRAPS-based edge gateway provides an efficient framework for improving network performance. The simulation-based approach enables comprehensive analysis of different scenarios, demonstrating the impact of scheduling, routing, and transmission parameters on network behavior.

CHAPTER 4

IMPLEMENTATION AND SIMULATION SETUP

4.1 Introduction

This chapter describes the implementation of the proposed IoT-based Wireless Sensor Network using the NetSim simulation tool. It includes the configuration of the network, selection of devices, traffic setup, routing protocols, and performance evaluation parameters. The implementation focuses on analyzing network behavior under different configurations and validating the effectiveness of the proposed edge-based gateway system.

4.2 Simulation Tool

The proposed system is implemented using NetSim (Version 14.4), a network simulation tool that provides a realistic environment for modeling IoT-based Wireless Sensor Networks. NetSim allows users to design network topologies, configure devices, assign traffic, and analyze performance metrics through graphical plots and logs. The Internet of Things (IoT) network module is selected in NetSim to create the simulation environment. This module supports wireless sensor nodes, gateways, routers, and cloud communication, enabling accurate modeling of real-world IoT systems.

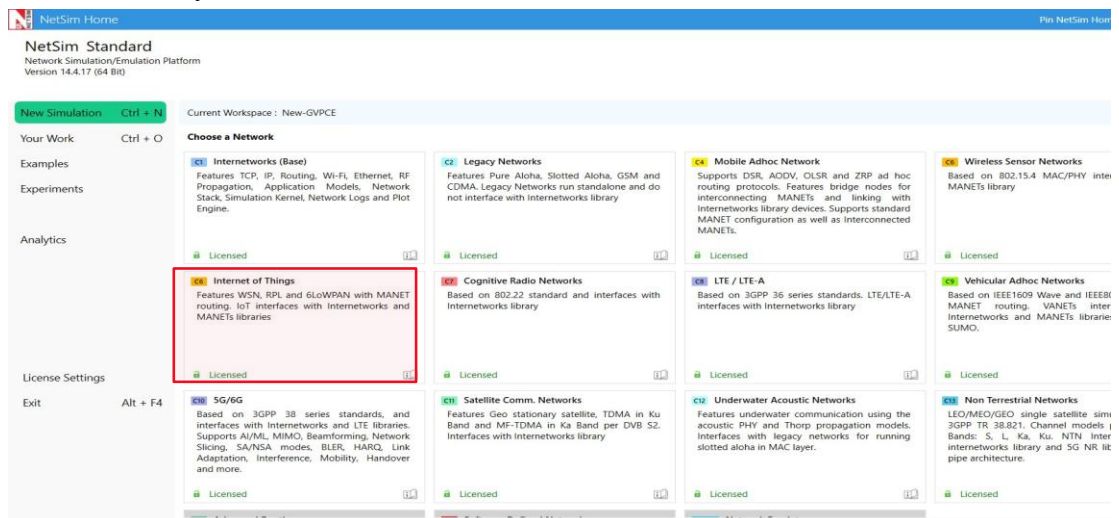


Figure 4.1: NetSim IoT Simulation Environment

4.3 Network Topology Setup

The simulation consists of multiple wireless sensor nodes connected to a gateway, which is further connected to a router and a cloud server. The gateway acts as a central node that collects data from all sensors and forwards it to the cloud. Sensor nodes are deployed using placement strategies such as uniform placement and random placement. In uniform placement, nodes are arranged in a structured grid pattern, ensuring equal spacing and controlled network behaviour. In this project, a network of multiple sensor nodes is deployed in a defined simulation area, forming a Wireless Sensor Network connected to the gateway. The gateway is automatically added in the NetSim workspace and serves as the main interface between the wireless network and the cloud infrastructure.

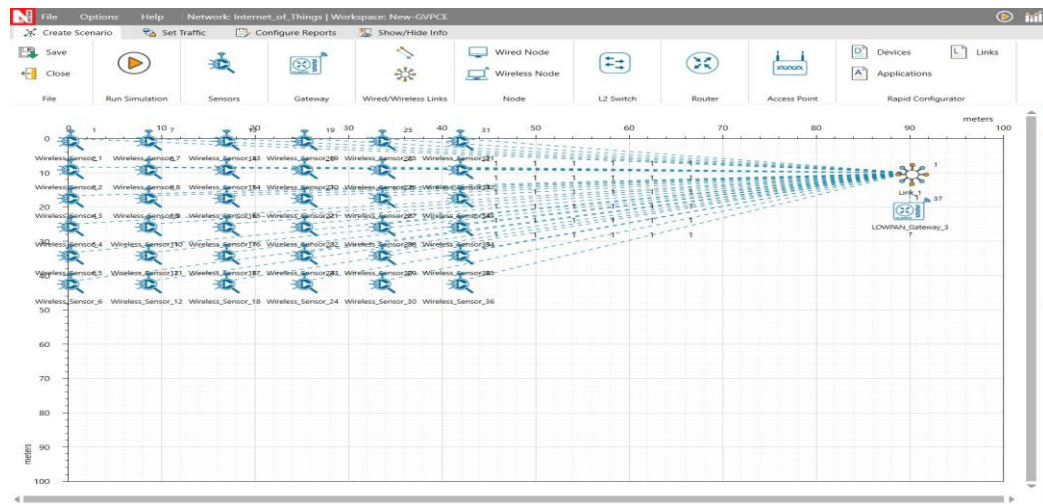


Figure 4.2: Uniform Placement of Sensor Nodes

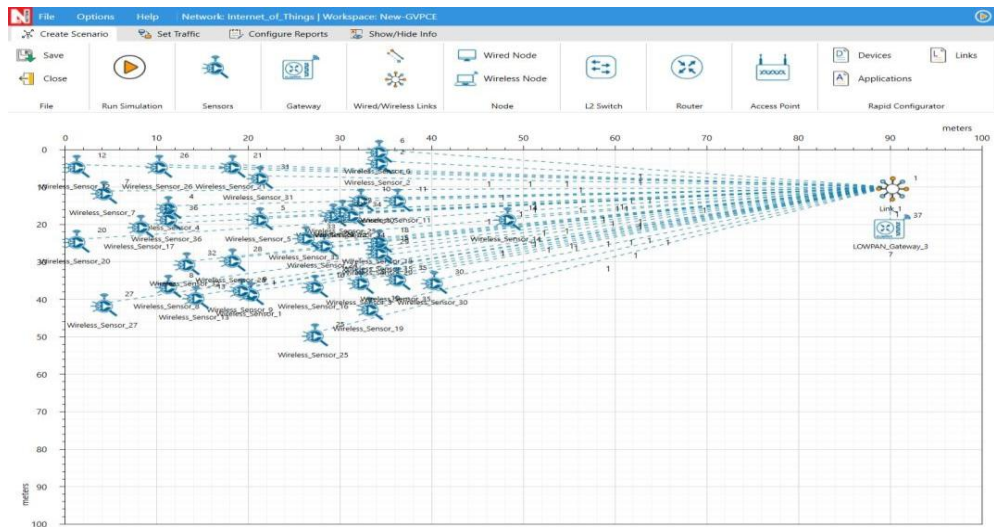


Figure 4.2: Random Placement of Sensor Nodes

4.4 Device Configuration

Each sensor node in the network is configured with specific parameters that define its behavior. These include general properties, application settings, transport layer configurations, and network layer parameters. The power settings of the sensor nodes are modified from the default main power supply to battery mode to simulate real-world IoT conditions. The energy model allows monitoring of energy consumption and remaining battery levels during simulation. The gateway is configured with routing protocols at the network layer, where either AODV or RPL can be selected. These protocols control how data packets are transmitted from sensor nodes to the cloud.

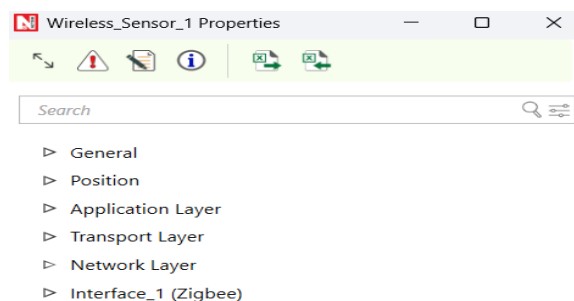


Figure 4.5: Sensor Node Configuration

4.5 Traffic Configuration

The network traffic is configured using the Constant Bit Rate (CBR) model. In this configuration, each sensor node generates data packets at a constant rate, simulating continuous data transmission in IoT applications. The traffic parameters include source and destination nodes, packet size, transmission interval, start time, and stop time. These parameters are defined in the application configuration settings of NetSim. Quality of Service (QoS) settings are also applied to classify traffic into different priority levels such as Best Effort (BE), Real-Time Polling Service (RTPS), and Unsolicited Grant Service (UGS). These classifications help in analyzing the performance of different scheduling mechanisms under varying traffic conditions.

NetSim - Rapid Application Configurator

New Application Delete Selected Export list to Excel Import list from Excel Close

Configure traffic/applications between devices in a tabular view. Editable fields are in blue. For bulk changes, use 'Export list to Excel', modify, and then import modified Spreadsheet using 'Import list from Excel'.

CBR 36	CUSTOM 0	VOICE 0	VIDEO 0	FTP 0	HTTP 0	EMAIL 0	DATABASE 0	GAMING 0										
	Application ID	Application Name	Source ID	Destination ID	Start Time (s)	End Time (s)	Transport protocol	Packet Size (B)	Interarrival Time (us)	Generation rate								
☐	1	App1_CBR	1	39	0	100000	UDP	1460	20000	584 kbps								
☐	2	App2_CBR	2	39	0	100000	UDP	1460	20000	584 kbps								
☐	3	App3_CBR	3	39	0	100000	UDP	1460	20000	584 kbps								
☐	4	App4_CBR	4	39	0	100000	UDP	1460	20000	584 kbps								
☐	5	App5_CBR	5	39	0	100000	UDP	1460	20000	584 kbps								
☐	6	App6_CBR	6	39	0	100000	UDP	1460	20000	584 kbps								
☐	7	App7_CBR	7	39	0	100000	UDP	1460	20000	584 kbps								
☐	8	App8_CBR	8	39	0	100000	UDP	1460	20000	584 kbps								
☐	9	App9_CBR	9	39	0	100000	UDP	1460	20000	584 kbps								
☐	10	App10_CBR	10	39	0	100000	UDP	1460	20000	584 kbps								
☐	11	App11_CBR	11	39	0	100000	UDP	1460	20000	584 kbps								
☐	12	App12_CBR	12	39	0	100000	UDP	1460	20000	584 kbps								
☐	13	App13_CBR	13	39	0	100000	UDP	1460	20000	584 kbps								
☐	14	App14_CBR	14	39	0	100000	UDP	1460	20000	584 kbps								
☐	15	App15_CBR	15	39	0	100000	UDP	1460	20000	584 kbps								
☐	16	App16_CBR	16	39	0	100000	UDP	1460	20000	584 kbps								
☐	17	App17_CBR	17	39	0	100000	UDP	1460	20000	584 kbps								
☐	18	App18_CBR	18	39	0	100000	UDP	1460	20000	584 kbps								
☐	19	App19_CBR	19	39	0	100000	UDP	1460	20000	584 kbps								
☐	20	App20_CBR	20	39	0	100000	UDP	1460	20000	584 kbps								
☐	21	App21_CBR	21	39	0	100000	UDP	1460	20000	584 kbps								
☐	22	App22_CBR	22	39	0	100000	UDP	1460	20000	584 kbps								
☐	23	App23_CBR	23	39	0	100000	UDP	1460	20000	584 kbps								
☐	24	App24_CBR	24	39	0	100000	UDP	1460	20000	584 kbps								
☐	25	App25_CBR	25	39	0	100000	UDP	1460	20000	584 kbps								
☐	26	App26_CBR	26	39	0	100000	UDP	1460	20000	584 kbps								

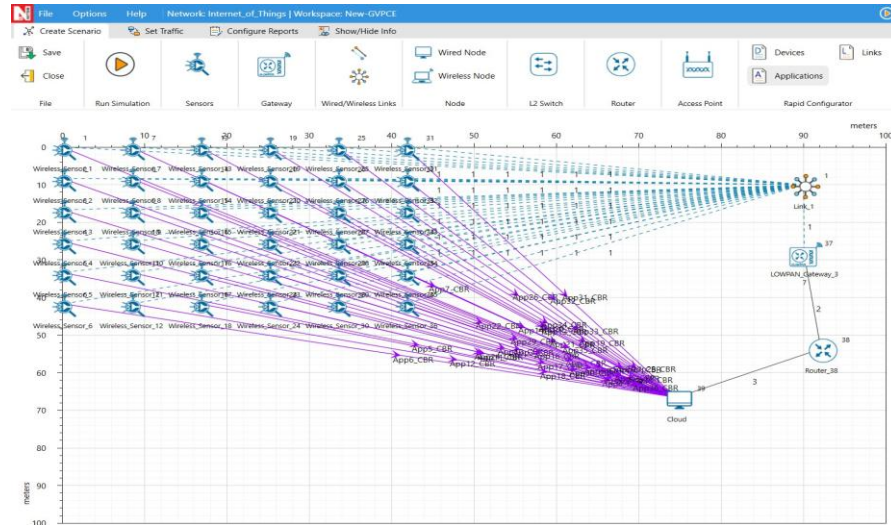


Figure 4.4,4.5: Traffic Configuration using CBR

4.6 Routing Protocol Configuration

The simulation evaluates two routing protocols: AODV and RPL. The routing protocol is configured at the gateway, and the selected protocol is applied to all sensor nodes in the network. AODV establishes routes dynamically when required, reducing overhead in dynamic environments. RPL, on the other hand, is specifically designed for IoT systems and supports low-power and lossy networks through a hierarchical routing structure. The performance of these protocols is analyzed under identical network conditions to compare their efficiency in terms of delay, throughput, and energy consumption.

4.7 Scheduling and Queuing Setup

The gateway is configured with different scheduling and queuing mechanisms to manage packet transmission efficiently. Scheduling techniques such as FIFO and Priority scheduling are implemented to analyze their effect on packet delivery order. FIFO processes packets in the order of arrival, while Priority scheduling gives preference to high-priority traffic. Queuing mechanisms such as Drop Tail are used to handle packet congestion. In Drop Tail, packets are

dropped only when the queue becomes full. These configurations help in studying network behavior under heavy traffic conditions.

4.8 Simulation Parameters and Scenarios

The system is evaluated under multiple scenarios to understand network performance:

- In the first scenario, all sensor nodes are configured with identical parameters, including packet size and transmission timing.
- In the second scenario, priority-based scheduling is applied to analyze its impact on packet delivery.
- In the third scenario, the start time of sensor nodes is varied to control packet transmission order.

These scenarios help in identifying the limitations of traditional scheduling methods and the importance of controlled transmission in wireless networks.

4.9 Performance Metrics Configuration

To evaluate system performance, various plots and logs are enabled in NetSim. These include:

- Throughput vs Time to measure data transmission efficiency
- End-to-End Delay vs Time to analyze latency
- Energy Consumption and Remaining Energy to monitor battery usage
- Packet Trace and Event Trace to analyze packet flow and network events

These metrics provide detailed insights into the behavior of the network under different configurations.

4.10 Implementation of Edge-Based Processing

The edge-based processing mechanism is implemented at the gateway level using the TRAPS approach. The gateway processes incoming data, filters unnecessary information, and transmits only relevant data to the cloud. This implementation reduces network load and improves response time. By combining edge computing with intelligent scheduling, the system achieves better performance compared to traditional cloud-based approaches.

4.11 Summary

This chapter presented the implementation details and simulation setup of the proposed IoT system using NetSim. The configuration of network topology, traffic models, routing protocols, and scheduling techniques provides a comprehensive framework for analyzing system performance. The simulation-based approach enables detailed evaluation of different scenarios, demonstrating the effectiveness of the proposed edge-based gateway in improving network efficiency.

CHAPTER 5

HARDWARE IMPLEMENTATION

5.1 Introduction

This chapter describes the implementation of the IoT-based gateway system using both simulation and real-time hardware. The system focuses on efficient data acquisition, processing, and transmission.

The implementation includes simulation using NetSim to evaluate performance and a hardware setup with Raspberry Pi, Arduino, and sensors for practical validation. Sensor data is collected by the Arduino, processed by the Raspberry Pi gateway, and uploaded to the cloud for remote monitoring via a web interface.

Overall, the integrated components demonstrate the feasibility and effectiveness of the proposed system in real-world applications.

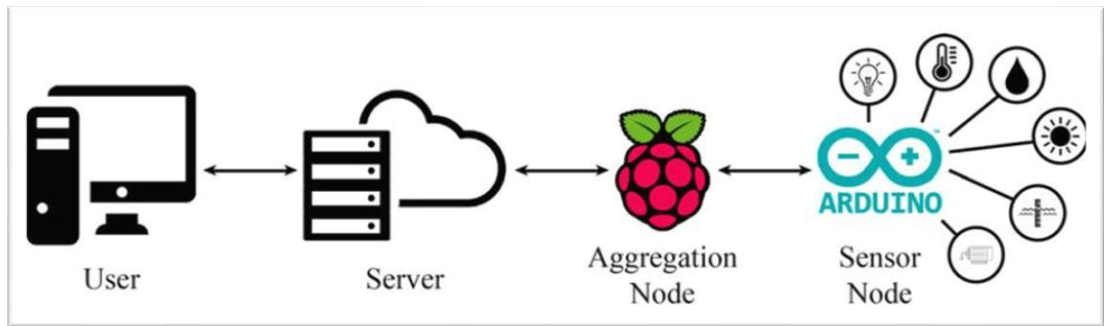


Figure 5.1: IoT Gateway System Architecture

5.2 Sensors

The proposed system uses six sensors—four DHT11 and two MQ3—to collect real-time environmental data. These sensors are connected to the Arduino, which acts as the data acquisition unit.

The DHT11 sensors measure temperature and humidity, while the MQ3 sensors detect alcohol/gas levels. The collected data is processed by the Arduino and sent to the Raspberry Pi for further processing and cloud integration, enabling real-time monitoring.

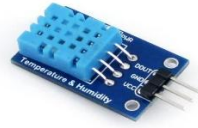


Figure 5.2.1: DHT11 sensors



Figure 5.2.2: MQ3 sensors

Sensor	Parameter	Specification
DHT11 (×4)	Temperature Range	0°C to 50°C
	Humidity Range	20% to 90% RH
	Accuracy	±2°C, ±5% RH
	Operating Voltage	3.3V – 5V
	Output	Digital
MQ3 (×2)	Detection	Alcohol (Ethanol Vapor)
	Detection Range	0.05 mg/L to 10 mg/L
	Operating Voltage	5V
	Sensitivity	High
	Output	Analog & Digital

Table 5.2.1: Specifications of Sensors Used in the System

Sensor Integration in System

- 4 × DHT11 sensors → Temperature & Humidity
- 2 × MQ3 sensors → Alcohol/Gas detection
- All sensors are connected to Arduino UNO
- Arduino collects and processes real-time data
- Data is transmitted to Raspberry Pi via serial communication

Data Flow:

Sensors → Arduino → RaspberryPi → Cloud

5.3 Arduino (Data Acquisition Unit)

The Arduino Uno serves as the data acquisition unit, acting as an interface between sensors and the gateway. It collects real-time data from four DHT11 and two MQ3 sensors, processes it, and formats it for transmission to the Raspberry Pi. DHT11 sensors provide digital data read through digital pins, while MQ3 sensors generate analog signals that are converted into digital values using the Arduino's ADC.



Figure 5.3: Arduino

Parameter	Specification
Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage	7–12V
Digital I/O Pins	14
Analog Input Pins	6
Clock Speed	16 MHz
Communication	Serial (USB)

Table 5.2.2: Arduino Specifications

Sensor	Connection Type	Arduino Pins
DHT11 (×4)	Digital Input	D2 – D5
MQ3 (×2)	Analog Input	A0, A1

Table 5.3: Arduino Pin Configuration

5.3.1 Arduino Program for Sensor Data Collection

Code:

```
#include <DHT.h>
#define DHTPIN1 2
#define DHTPIN2 3
#define DHTPIN3 4
#define DHTPIN4 5
#define MQ3_1 A0
#define MQ3_2 A1
#define DHTTYPE DHT11
DHT dht1(DHTPIN1, DHTTYPE);
DHT dht2(DHTPIN2, DHTTYPE);
DHT dht3(DHTPIN3, DHTTYPE);
DHT dht4(DHTPIN4, DHTTYPE);
void setup() {
  Serial.begin(115200);
  dht1.begin();
  dht2.begin();
  dht3.begin();
  dht4.begin();
}
void loop() {
  float temp1 = dht1.readTemperature();
  float temp2 = dht2.readTemperature();
  float hum1 = dht3.readHumidity();
  float hum2 = dht4.readHumidity();
  int gas1 = analogRead(MQ3_1);
  int gas2 = analogRead(MQ3_2);
  Serial.print("TEMP1:"); Serial.print(temp1);
  Serial.print(",TEMP2:"); Serial.print(temp2);
  Serial.print(",HUM1:"); Serial.print(hum1);
```

```
Serial.print(",HUM2:"); Serial.print(hum2);  
Serial.print(",GAS1:"); Serial.print(gas1);  
Serial.print(",GAS2:"); Serial.println(gas2);  
delay(3000);  
}
```

Working and Role of Arduino

The Arduino Uno acts as the data acquisition unit, interfacing sensors with the gateway. It reads digital data from DHT11 sensors and analog data from MQ3 sensors, converting analog signals using its ADC. The data is processed, formatted, and sent to the Raspberry Pi via USB for real-time monitoring.

It also performs initial processing and ensures efficient communication between sensors and the gateway.

5.4 Raspberry Pi 4 (Gateway)

The Raspberry Pi 4 acts as the gateway, connecting the sensor network to the cloud. It receives data, processes it, evaluates conditions, and prepares it for transmission.

This reduces unnecessary data transfer and improves efficiency. The data is sent to Google Sheets for real-time monitoring, and remote access tools like VNC enable control from anywhere.



Figure 5.4: Raspberry Pi 4

Parameter	Specification
Processor	Quad-core ARM Cortex-A72
RAM	2GB / 4GB / 8GB
Operating System	Raspberry Pi OS
Connectivity	Wi-Fi, Bluetooth, Ethernet
USB Ports	USB 2.0 / USB 3.0
GPIO Pins	40 Pins
Power Supply	5V

Table 5.4: Raspberry Pi 4 Specifications

5.4.1 Raspberry Pi Program for Gateway Processing

Code:

```

import serial
import time
import requests
SERIAL_PORT = '/dev/ttyACM0'
BAUD_RATE = 115200
WEB_APP_URL =
"https://script.google.com/macros/s/AKfycbzBUXVzQMAtaf3ZwkjqpJxeoz5bSB2B3lovf42ST6FWk
j-IACup8rHfOnMg9jkOAowkQ/exec"
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
def classify(sensor, value):
    if "TEMP" in sensor:
        if value > 34:
            return "RED"
        elif value > 30:
            return "ORANGE"
        else:
            return "GREEN"
    elif "HUM" in sensor:
        if value > 95:
            return "ORANGE"
        elif value > 85:
            return "GREEN"
        else:
            return "GREEN"
    elif "GAS" in sensor:
        if value > 400:
            return "RED"
        elif value > 200:

```

```

        return "ORANGE"
    else:
        return "GREEN"
def process_data(line):
    parts = line.split(',')
    readings = {}
    for part in parts:
        key, val = part.split(':')
        readings[key] = float(val)
    temps = {k: v for k, v in readings.items() if "TEMP" in k}
    hums = {k: v for k, v in readings.items() if "HUM" in k}
    gases = {k: v for k, v in readings.items() if "GAS" in k}
    temps = sorted(temps.items(), key=lambda x: x[1], reverse=True)
    hums = sorted(hums.items(), key=lambda x: x[1], reverse=True)
    gases = sorted(gases.items(), key=lambda x: x[1], reverse=True)
    return temps + hums + gases
def send_to_sheet(sensor, value, status):
    data = {
        "sensor": sensor,
        "value": value,
        "status": status
    }
    try:
        response = requests.post(WEB_APP_URL, json=data)
        print("Sent:", data, "Response:", response.status_code)
    except Exception as e:
        print("Send error:", e)

while True:
    try:
        line = ser.readline().decode('utf-8').strip()

        if not line:
            continue

        print("Raw:", line)

        sorted_data = process_data(line)

        for sensor, value in sorted_data:
            status = classify(sensor, value)
            send_to_sheet(sensor, value, status)

        time.sleep(5)

    except Exception as e:
        print("Error:", e)

```

Data Processing at Gateway

- The gateway (Raspberry Pi) reads serial data from Arduino.
- Incoming data is parsed and separated into:
 - Temperature readings
 - Humidity readings
 - Gas readings
- Sensor values are **sorted in descending order** to prioritize critical data.
- Enables efficient handling and transmission of important data first.

Data Classification Mechanism

- Sensor values are classified into three levels:
 - **GREEN (Normal Condition)**
 - **ORANGE (Moderate Condition)**
 - **RED (Critical Condition)**

Threshold Conditions:

- **Temperature**
 - 34 → RED
 - 30–34 → ORANGE
 - < 30 → GREEN
- **Humidity**
 - 95 → ORANGE
 - 85–95 → GREEN
 - < 85 → GREEN
- **Gas (MQ-3)**
 - 400 → RED
 - 200–400 → ORANGE
 - < 200 → GREEN

Working and Role as Gateway

The Raspberry Pi 4 functions as an intelligent gateway by receiving sensor data from the Arduino and performing local processing before forwarding it to the cloud. It filters, formats, and analyzes the incoming data, ensuring only relevant and structured information is transmitted. By acting as a gateway, the Raspberry Pi reduces network load, improves data management, and enables real-time communication between the sensor layer and cloud layer. It also supports remote access and monitoring, making the system scalable and efficient for IoT applications.

5.5 Communication Protocols Used in Hardware

The hardware implementation of the IoT-based gateway system uses multiple communication protocols for efficient data transfer between sensors, Arduino, Raspberry Pi, and the cloud. UART serial communication is used between Arduino and Raspberry Pi, where sensor data from DHT11 and MQ3 is transmitted via USB. The Raspberry Pi sends data to Google Sheets using HTTP, enabling real-time cloud storage and access. Communication over the internet relies on TCP/IP for reliable data transmission, while Wi-Fi (IEEE 802.11) provides network connectivity. At the sensor level, DHT11 uses digital signals, and MQ3 uses analog signals converted via the Arduino's ADC. Overall, these protocols ensure smooth data flow and real-time monitoring in the system.

5.6 Remote Access using RealVNC Viewer

Remote access to the Raspberry Pi is achieved using RealVNC Viewer, which allows the user to control and monitor the gateway device from a remote system. This eliminates the need for a dedicated monitor, keyboard, or mouse connected to the Raspberry Pi. RealVNC Viewer enables the Raspberry Pi desktop to be accessed from a laptop or desktop over a network connection. The connection is established using the IP address of the Raspberry Pi, and communication is carried out over Wi-Fi within the same network.

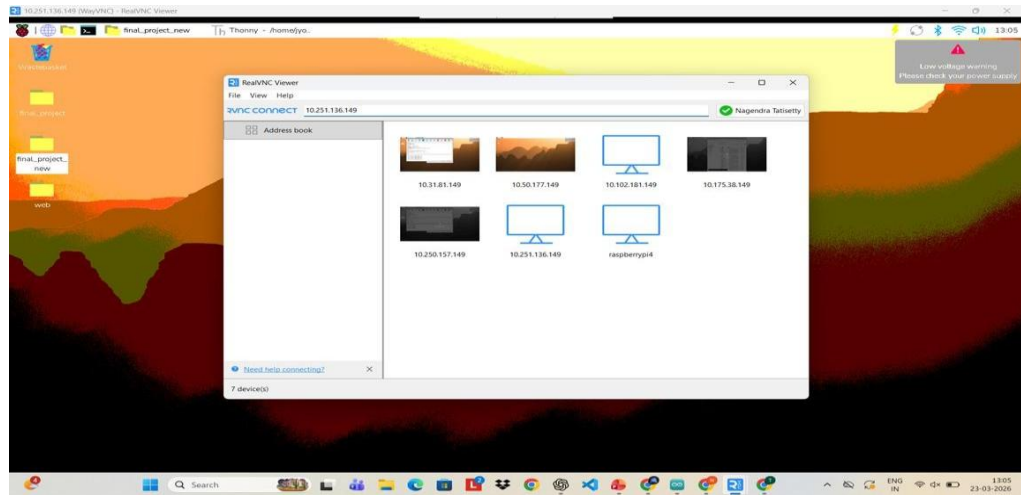


Figure 5.6: Remote Access of Raspberry Pi using RealVNC Viewer

Working and Role of RealVNC Viewer

RealVNC Viewer acts as a remote interface for accessing the Raspberry Pi. Once the connection is established, the user can view the Raspberry Pi desktop, execute programs, monitor sensor data, and manage files in real time. It plays an important role in the system by enabling remote operation of the gateway without requiring physical interaction. This improves system flexibility and allows easy monitoring and control from any connected device.

Features of RealVNC Viewer

- Provides **remote access** to the Raspberry Pi from another device
- Supports **wireless connectivity** through Wi-Fi network
- Offers a **graphical user interface** for viewing the Raspberry Pi desktop
- Allows users to **execute programs and monitor output** in real time
- Eliminates the need for **external peripherals** such as monitor, keyboard, and mouse

5.7 Cloud Storage using Google Sheets

Google Sheets is used as the cloud storage platform in the proposed system. It acts as a centralized database where the sensor data collected from the hardware setup is stored and maintained in real time. The Raspberry Pi gateway uploads the processed sensor data to Google

Sheets using internet connectivity. Each data entry includes important parameters such as timestamp, sensor name, sensor values, and status (NORMAL or CRITICAL). This enables continuous data logging and easy access for monitoring and analysis

Google Sheets Link

<https://docs.google.com/spreadsheets/d/17HXaifcoGC6UmGbrgZBy4IKfU0NR5lIDpzSM0FHpDQs/edit?usp=sharing>

5.7.1 Google Sheets Script / API Code

```
function doPost(e) {
  var sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Sheet1");

  var data = JSON.parse(e.postData.contents);

  // Insert at top
  sheet.insertRowBefore(2);
  sheet.getRange(2, 1, 1, 3).setValues([
    [data.sensor, data.value, data.status]
  ]);

  // Limit to 600 rows
  var lastRow = sheet.getLastRow();
  if (lastRow > 600) {
    sheet.deleteRow(lastRow);
  }

  return ContentService
    .createTextOutput(JSON.stringify({ "status": "success" }))
    .setMimeType(ContentService.MimeType.JSON);
}

// API for website
function doGet() {
  var sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Sheet1");
  var data = sheet.getDataRange().getValues();

  var result = [];

  for (var i = 1; i < data.length; i++) {
    result.push({
      sensor: data[i][0],
      value: data[i][1],
      status: data[i][2]
    });
  }

  return ContentService
```

```

        .createTextOutput(JSON.stringify(result))
        .setMimeType(ContentService.MimeType.JSON);
    }

```

Data Structure in Google Sheets

- The system uses a **structured JSON format** for communication between the gateway, cloud (Google Sheets), and web application.

Cloud Communication

- Data is sent to Google Sheets using **HTTP POST requests**.
- Each data packet includes:
 - Sensor name
 - Sensor value
 - Status (Normal/Critical)

Google Apps Script acts as an API endpoint for data storage

1. Incoming Data Format (POST Request)

Data sent from Raspberry Pi to Google Sheets follows this JSON structure:

```

{
  "sensor": "TEMP1",
  "value": 32.5,
  "status": "GREEN"
}

```

2. Outgoing Data Format (GET API Response)

Data retrieved by the web application is returned as a JSON array:

```

[
  {

```

```

    "sensor": "TEMP1",
    "value": 32.5,
    "status": "GREEN"
  },
  {
    "sensor": "GAS1",
    "value": 420,
    "status": "RED"
  }
]

```

Working and Role of Google Sheets

Google Sheets functions as a cloud-based storage system that receives data from the Raspberry Pi and stores it in a structured tabular format. The data is updated automatically at regular intervals, ensuring real-time synchronization. It plays a crucial role in the system by enabling data persistence, easy visualization, and remote accessibility. The stored data can be accessed from any device with internet connectivity and can also be integrated with web applications for display and analysis.

Features of Google Sheets

- Stores sensor data in **real-time**
- Provides **timestamp-based data logging**
- Allows **remote access** from any device
- Supports **easy visualization and analysis**
- Acts as a **lightweight cloud database**

Data Flow

Sensors → Arduino → Raspberry Pi → **Google Sheets (Cloud Storage)**

5.8 Web Application (User Interface)

Website Link (GitHub Pages)

<https://322103312207-project.github.io/8th-sem-project/>

5.8.1 Source Code (HTML / CSS / JavaScript)

Code:

```
<!DOCTYPE html>

<html>

<head>

<title>Sensor Dashboard</title>

<style>

  body { font-family: Arial; background:#111; color:white; }

  .card {

    padding: 12px;

    margin: 10px;

    border-radius: 8px;

    font-size: 18px;

  }

  .RED { background: red; }

  .ORANGE { background: orange; }

  .GREEN { background: green; }

</style>

</head>

<body>

<h2>Live Sensor Data</h2>
```

```

<div id="data"></div>

<script>

const API_URL =
"https://script.google.com/macros/s/AKfycbzBUXVzQMcAtaf3ZwkjqpJxeoz5bSB2B3lovf42ST6FWk
j-IACup8rHfOnMg9jkOAowkQ/exec";

function loadData() {

  fetch(API_URL)

    .then(res => res.json())

    .then(data => {

      let container = document.getElementById("data");

      container.innerHTML = "";

      data.reverse().forEach(row => {

        let div = document.createElement("div");

        div.className = "card " + row.status;

        div.innerHTML = `

          <b>${row.sensor}</b><br>

          Value: ${row.value}<br>

          Status: ${row.status}

        `;

        container.appendChild(div);

      });

    });

  setInterval(loadData, 5000);

```

```
loadData();
```

```
</script>
```

```
</body>
```

```
</html>
```

Data Integration

- The system retrieves sensor data from **Google Sheets**, which acts as cloud storage.
- Data is transmitted from the gateway to the cloud using HTTP requests.
- The web application fetches and displays **real-time sensor readings**.
- Ensures continuous synchronization between hardware sensors and cloud database.

Web Application Features

- Displays **real-time sensor values** from the cloud.
- Shows environmental parameters such as:
 - Temperature (DHT11 Sensors)
 - Humidity (DHT11 Sensors)
 - Gas concentration (MQ-3 Sensors)
- Includes **timestamp and status indicators** (GREEN / ORANGE / RED).
- Provides a **user-friendly interface** for easy monitoring.
- Accessible from any device (mobile/laptop) via a web browser.

Output Snapshot (Website)

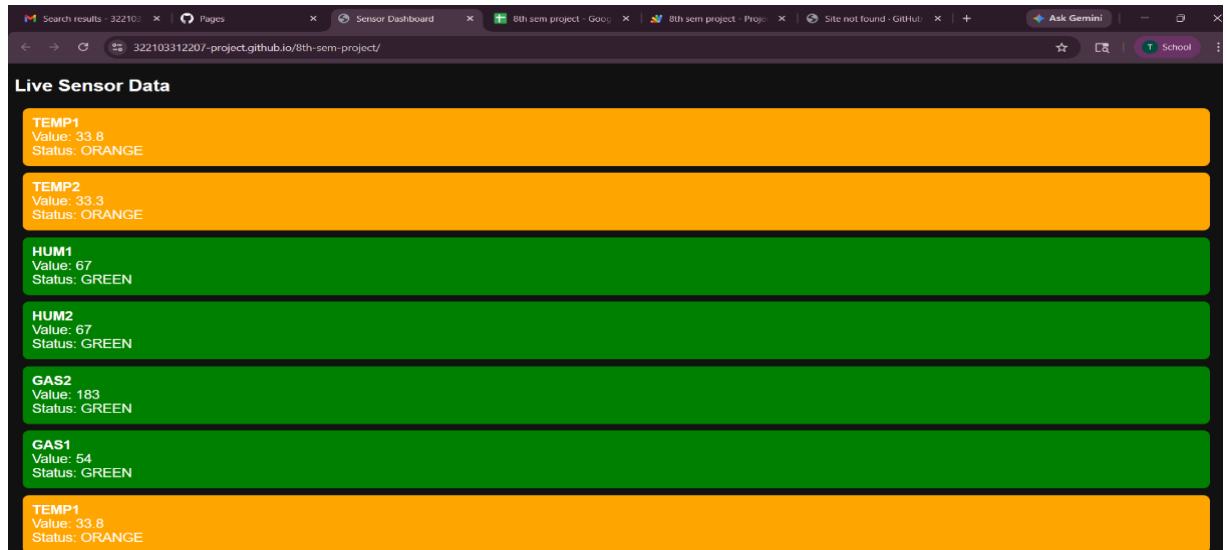


Figure 5.8: Web Application Output Displaying Live Sensor Data

Data Flow

Sensors (Data Collection) → **Arduino** (Data Acquisition & Processing) → **Raspberry Pi** (Gateway & Data Processing) → **Google Sheets** (Cloud Storage & Logging) → **Website** (User Interface & Visualization)

5.9 Summary

This chapter describes the hardware implementation of the proposed IoT gateway system using sensors, Arduino, and Raspberry Pi. The sensors collect real-time data such as temperature, humidity, and gas levels, which are sent to the Raspberry Pi via Arduino. The Raspberry Pi processes and classifies the data into GREEN, ORANGE, and RED levels based on predefined thresholds. The processed data is then transmitted to Google Sheets using HTTP (Google Apps Script API) for cloud storage. A web application retrieves and displays this data in real time with sensor values, status indicators, and timestamps, enabling remote monitoring. Overall, the system demonstrates efficient data collection, processing, and cloud-based monitoring, making it a scalable IoT solution.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Introduction

This chapter presents the simulation results of the proposed IoT-based WSN using NetSim. The system is evaluated under different scenarios to analyze routing, scheduling, and edge processing. Performance is measured using metrics like throughput, delay, energy consumption, and packet delivery ratio, showing the effectiveness of the proposed system.

6.2 Case 1: Equal Configuration of Sensor Nodes

In the first scenario, all sensor nodes use identical parameters, creating uniform traffic. However, packet arrival at the cloud is not sequential and occurs randomly. This is due to the contention-based wireless medium, where nodes compete for channel access, causing variations in throughput and delay.

6.3 Case 2: Priority and FIFO Scheduling

In the second scenario, priority levels and FIFO scheduling are applied, expecting higher-priority nodes to transmit first. However, packet delivery remains unpredictable. This is because wireless communication uses contention-based access, where factors like random backoff and collisions affect transmission, causing delays even for high-priority packets.

6.4 Case 3: Controlled Start Time

In the third scenario, the number of sensor nodes is reduced, and the transmission start times are varied while keeping other parameters constant. This approach is used to analyze the effect of controlled transmission on network behavior. The results show that sensor nodes with earlier start times successfully transmit their data before others, resulting in a predictable packet arrival order at the cloud. This demonstrates that controlling transmission timing is an effective method

for managing packet delivery in wireless networks. The gateway behavior in this scenario aligns with the configured parameters, indicating improved network control and stability compared to previous cases.

6.5 Throughput Analysis

Throughput represents the rate of successful data transmission in the network. The simulation results show that throughput varies among sensor nodes due to differences in channel access and transmission delays. In scenarios with identical configurations, throughput fluctuations are observed due to contention and interference in the wireless medium. However, with controlled transmission parameters, throughput becomes more stable and predictable.

Applicatio	Applicatio	Source ID	Destinatio	Generatio	Packets G	Packets R	Payload g	Payload r	Throughp	Delay (mi	Jitter (microsecond)
1	App1_CBR	1	39	0.584	225000	358	21900000	35240	0.00094	1.46E+08	769222.3
2	App2_CBR	2	39	0.584	225000	359	21900000	35140	0.000937	1.39E+08	768374
3	App3_CBR	3	39	0.584	225000	335	21900000	32740	0.000873	1.39E+08	820584.4
4	App4_CBR	4	39	0.584	225000	353	21900000	34500	0.00092	1.39E+08	781651.6
5	App5_CBR	5	39	0.584	225000	355	21900000	34740	0.000926	1.36E+08	776557.3
6	App6_CBR	6	39	0.584	225000	338	21900000	32960	0.000879	1.35E+08	812793
7	App7_CBR	7	39	0.584	225000	328	21900000	32200	0.000859	1.33E+08	830473.7
8	App8_CBR	8	39	0.584	225000	348	21900000	33920	0.000905	1.36E+08	789844.4
9	App9_CBR	9	39	0.584	225000	376	21900000	36560	0.000975	1.31E+08	728509.1
10	App10_CBR	10	39	0.584	225000	342	21900000	33560	0.000895	1.34E+08	807836.2
11	App11_CBR	11	39	0.584	225000	371	21900000	36380	0.00097	1.38E+08	740932.4
12	App12_CBR	12	39	0.584	225000	343	21900000	33180	0.000885	1.36E+08	803044.8
13	App13_CBR	13	39	0.584	225000	340	21900000	33160	0.000884	1.39E+08	811753.6
14	App14_CBR	14	39	0.584	225000	329	21900000	32300	0.000861	1.42E+08	838573.9
15	App15_CBR	15	39	0.584	225000	362	21900000	35600	0.000949	1.35E+08	757078.5
16	App16_CBR	16	39	0.584	225000	368	21900000	36040	0.000961	1.32E+08	748708.2
17	App17_CBR	17	39	0.584	225000	389	21900000	37780	0.001007	1.35E+08	708104.5
18	App18_CBR	18	39	0.584	225000	325	21900000	31740	0.000846	1.4E+08	844764.1
19	App19_CBR	19	39	0.584	225000	353	21900000	34300	0.000915	1.4E+08	778323.9
20	App20_CBR	20	39	0.584	225000	344	21900000	33440	0.000892	1.41E+08	800094.1
21	App21_CBR	21	39	0.584	225000	335	21900000	32340	0.000862	1.39E+08	820593.1
22	App22_CBR	22	39	0.584	225000	366	21900000	35720	0.000953	1.38E+08	746389.2
23	App23_CBR	23	39	0.584	225000	334	21900000	32400	0.000864	1.34E+08	825893.1
24	App24_CBR	24	39	0.584	225000	374	21900000	36200	0.000965	1.48E+08	734846.6
25	App25_CBR	25	39	0.584	225000	357	21900000	34620	0.000923	1.38E+08	772134.1
26	App26_CBR	26	39	0.584	225000	333	21900000	32740	0.000873	1.41E+08	828210.8
27	App27_CBR	27	39	0.584	225000	350	21900000	34280	0.000914	1.4E+08	784693.4
28	App28_CBR	28	39	0.584	225000	382	21900000	37280	0.000994	1.39E+08	717637.3
29	App29_CBR	29	39	0.584	225000	336	21900000	32920	0.000878	1.39E+08	818214.1
30	App30_CBR	30	39	0.584	225000	367	21900000	35900	0.000957	1.47E+08	743605.8
31	App31_CBR	31	39	0.584	225000	334	21900000	32560	0.000868	1.32E+08	827443.2
32	App32_CBR	32	39	0.584	225000	362	21900000	35480	0.000946	1.35E+08	763577.9
33	App33_CBR	33	39	0.584	225000	335	21900000	32740	0.000873	1.41E+08	818583.1
34	App34_CBR	34	39	0.584	225000	362	21900000	35200	0.000939	1.34E+08	759334.2
35	App35_CBR	35	39	0.584	225000	326	21900000	31800	0.000848	1.49E+08	847109.1
36	App36_CBR	36	39	0.584	225000	386	21900000	37600	0.001003	1.42E+08	713561.5

Table 6.5.1: AODV Application metrics

Rank	Application	Delay	Throughput
1	App9_CBR	1.31E+08	0.000975
2	App16_CBR	1.32E+08	0.000961
3	App31_CBR	1.32E+08	0.000868
4	App10_CBR	1.34E+08	0.000895

5	App23_CBR	1.34E+08	0.000864
6	App6_CBR	1.35E+08	0.000879
7	App17_CBR	1.35E+08	0.001007
8	App5_CBR	1.36E+08	0.000926
9	App8_CBR	1.36E+08	0.000905
10	App12_CBR	1.36E+08	0.000885
11	App2_CBR	1.39E+08	0.000937
12	App3_CBR	1.39E+08	0.000873
13	App4_CBR	1.39E+08	0.000920
14	App11_CBR	1.38E+08	0.000970
15	App15_CBR	1.35E+08	0.000949
16	App25_CBR	1.38E+08	0.000923
17	App33_CBR	1.41E+08	0.000873
18	App7_CBR	1.33E+08	0.000859
19	App14_CBR	1.42E+08	0.000861
20	App18_CBR	1.40E+08	0.000846
21	App19_CBR	1.40E+08	0.000915
22	App20_CBR	1.41E+08	0.000892
23	App21_CBR	1.39E+08	0.000862
24	App22_CBR	1.38E+08	0.000953
25	App24_CBR	1.48E+08	0.000965
26	App26_CBR	1.41E+08	0.000873
27	App27_CBR	1.40E+08	0.000914
28	App28_CBR	1.39E+08	0.000994

29	App29_CBR	1.39E+08	0.000878
30	App30_CBR	1.47E+08	0.000957
31	App32_CBR	1.35E+08	0.000946
32	App34_CBR	1.34E+08	0.000939
33	App35_CBR	1.49E+08	0.000848
34	App36_CBR	1.42E+08	0.001003
35	App1_CBR	1.46E+08	0.000940
36	App13_CBR	1.39E+08	0.000884

Table 6.5.2: Priority Order (Which sensor reach first to Cloud)

Applicatio	Applicatio	Source ID	Destinatio	Generatio	Packets G	Packets R	Payload g	Payload r	Throughp	Delay (mi	Jitter (microsecond)
1	App1_CBR	1	39	0.584	225000	311	21900000	30580	0.000815	1.34E+08	910291.5
2	App2_CBR	2	39	0.584	225000	311	21900000	30460	0.000812	1.43E+08	909458.3
3	App3_CBR	3	39	0.584	225000	323	21900000	31540	0.000841	1.44E+08	868472.3
4	App4_CBR	4	39	0.584	225000	336	21900000	32880	0.000877	1.43E+08	839117.3
5	App5_CBR	5	39	0.584	225000	302	21900000	29320	0.000782	1.34E+08	930675.2
6	App6_CBR	6	39	0.584	225000	293	21900000	28500	0.00076	1.32E+08	967091.5
7	App7_CBR	7	39	0.584	225000	323	21900000	31620	0.000843	1.3E+08	867247.9
8	App8_CBR	8	39	0.584	225000	356	21900000	34640	0.000924	1.33E+08	786128.9
9	App9_CBR	9	39	0.584	225000	301	21900000	29580	0.000789	1.38E+08	937794.4
10	App10_CB	10	39	0.584	225000	326	21900000	31760	0.000847	1.38E+08	858033
11	App11_CB	11	39	0.584	225000	315	21900000	30860	0.000823	1.28E+08	887247.5
12	App12_CB	12	39	0.584	225000	357	21900000	35100	0.000936	1.31E+08	789633.4
13	App13_CB	13	39	0.584	225000	358	21900000	34640	0.000924	1.36E+08	788771.4
14	App14_CB	14	39	0.584	225000	338	21900000	32880	0.000877	1.31E+08	826966.6
15	App15_CB	15	39	0.584	225000	343	21900000	33340	0.000889	1.34E+08	822142
16	App16_CB	16	39	0.584	225000	358	21900000	34760	0.000927	1.49E+08	784629.5
17	App17_CB	17	39	0.584	225000	321	21900000	31180	0.000831	1.26E+08	870478.2
18	App18_CB	18	39	0.584	225000	338	21900000	32920	0.000878	1.33E+08	832968.7
19	App19_CB	19	39	0.584	225000	351	21900000	34300	0.000915	1.35E+08	804070.9
20	App20_CB	20	39	0.584	225000	309	21900000	30220	0.000806	1.31E+08	910684.1
21	App21_CB	21	39	0.584	225000	329	21900000	32140	0.000857	1.31E+08	853090.2
22	App22_CB	22	39	0.584	225000	355	21900000	34860	0.00093	1.33E+08	790530.4
23	App23_CB	23	39	0.584	225000	326	21900000	31880	0.00085	1.34E+08	864943.9
24	App24_CB	24	39	0.584	225000	352	21900000	34320	0.000915	1.33E+08	797032.6
25	App25_CB	25	39	0.584	225000	331	21900000	32300	0.000861	1.34E+08	849387
26	App26_CB	26	39	0.584	225000	322	21900000	31640	0.000844	1.36E+08	867411.4
27	App27_CB	27	39	0.584	225000	372	21900000	36320	0.000969	1.45E+08	748212.6
28	App28_CB	28	39	0.584	225000	373	21900000	36380	0.00097	1.4E+08	752506.6
29	App29_CB	29	39	0.584	225000	323	21900000	31380	0.000837	1.37E+08	869782.9
30	App30_CB	30	39	0.584	225000	362	21900000	35200	0.000939	1.35E+08	774429.6
31	App31_CB	31	39	0.584	225000	369	21900000	35980	0.000959	1.36E+08	757022.4
32	App32_CB	32	39	0.584	225000	347	21900000	33940	0.000905	1.32E+08	810339.4
33	App33_CB	33	39	0.584	225000	324	21900000	31880	0.00085	1.37E+08	863959.9
34	App34_CB	34	39	0.584	225000	340	21900000	33000	0.00088	1.33E+08	821052.5
35	App35_CB	35	39	0.584	225000	345	21900000	33820	0.000902	1.35E+08	812338.9
36	App36_CB	36	39	0.584	225000	327	21900000	31740	0.000846	1.34E+08	853915.1

Table 6.5.3: RLP Application Metrics

Rank	Application	Delay (μs)	Throughput
1	App17_CBR	1.26E+08	0.000831

2	App11_CBR	1.28E+08	0.000823
3	App20_CBR	1.31E+08	0.000806
4	App21_CBR	1.31E+08	0.000857
5	App12_CBR	1.31E+08	0.000936
6	App14_CBR	1.31E+08	0.000877
7	App6_CBR	1.32E+08	0.000760
8	App32_CBR	1.32E+08	0.000905
9	App8_CBR	1.33E+08	0.000924
10	App22_CBR	1.33E+08	0.000930
11	App24_CBR	1.33E+08	0.000915
12	App34_CBR	1.33E+08	0.000880
13	App18_CBR	1.33E+08	0.000878
14	App1_CBR	1.34E+08	0.000815
15	App2_CBR	1.34E+08	0.000812
16	App5_CBR	1.34E+08	0.000782
17	App15_CBR	1.34E+08	0.000889
18	App23_CBR	1.34E+08	0.000850
19	App25_CBR	1.34E+08	0.000861
20	App36_CBR	1.34E+08	0.000846
21	App30_CBR	1.35E+08	0.000939
22	App35_CBR	1.35E+08	0.000902
23	App19_CBR	1.35E+08	0.000915
24	App13_CBR	1.36E+08	0.000924
25	App26_CBR	1.36E+08	0.000844

26	App31_CBR	1.36E+08	0.000959
27	App33_CBR	1.37E+08	0.000850
28	App29_CBR	1.37E+08	0.000837
29	App10_CBR	1.38E+08	0.000847
30	App7_CBR	1.38E+08	0.000843
31	App9_CBR	1.38E+08	0.000789
32	App28_CBR	1.40E+08	0.000970
33	App4_CBR	1.43E+08	0.000877
34	App3_CBR	1.44E+08	0.000841
35	App27_CBR	1.45E+08	0.000969
36	App16_CBR	1.49E+08	0.000927

Table 6.5.4: Priority Order (Which sensor reach first to Cloud)

6.6 End-to-End Delay Analysis

End-to-end delay measures the time taken for packets to travel from sensors to the cloud. Results show that delay varies across scenarios. In equal configuration, delay is inconsistent due to random transmission. In priority scheduling, improvement is limited due to wireless behavior. In controlled start time, delay is reduced and more consistent, highlighting better transmission control.

6.7 Energy Consumption Analysis

Energy consumption is a critical factor in Wireless Sensor Networks, as sensor nodes operate on limited battery power. The simulation results show that energy consumption varies depending on routing protocols and network conditions. For the AODV protocol, the energy efficiency of sensor nodes is observed to be approximately in the range of 99.88% to 99.90%,

indicating minimal energy consumption during operation. For the RPL protocol, the energy efficiency is slightly lower, ranging around 99.86% to 99.87%, due to additional routing overhead and control message exchanges. These results indicate that AODV performs slightly better in terms of energy efficiency, while RPL provides better adaptability for IoT environments.

Battery model								
Device Na	Initial ene	Consumed	Remainin	Harvested	Transmitt	Receiving	Idle energ	Sleep energy(mJ)
WIRELESS_3888000	4604.974	3883827	431.9999	1265.802	31.82727	3307.344	0	
WIRELESS_3888000	4611.884	3883820	431.9999	1273.927	32.37913	3305.577	0	
WIRELESS_3888000	4590.694	3883841	431.9999	1248.088	31.82174	3310.785	0	
WIRELESS_3888000	4592.485	3883840	431.9999	1250.602	31.46342	3310.42	0	
WIRELESS_3888000	4615.861	3883816	431.9999	1279.469	31.63374	3304.758	0	
WIRELESS_3888000	4609.195	3883823	431.9999	1271.579	31.16593	3306.45	0	
WIRELESS_3888000	4590.569	3883841	431.9999	1248.179	31.51983	3310.871	0	
WIRELESS_3888000	4579.683	3883852	431.9999	1234.289	31.98763	3313.406	0	
WIRELESS_3888000	4590.81	3883841	431.9999	1248.653	31.30307	3310.853	0	
WIRELESS_3888000	4587.454	3883845	431.9999	1244.349	31.47448	3311.63	0	
WIRELESS_3888000	4605.348	3883827	431.9999	1266.368	31.7023	3307.277	0	
WIRELESS_3888000	4581.951	3883850	431.9999	1237.215	31.8505	3312.886	0	
WIRELESS_3888000	4609.344	3883823	431.9999	1271.299	31.73659	3306.308	0	
WIRELESS_3888000	4576.205	3883856	431.9999	1230.392	31.47338	3314.34	0	
WIRELESS_3888000	4611.276	3883821	431.9999	1274.234	31.07525	3305.966	0	
WIRELESS_3888000	4611.053	3883821	431.9999	1273.354	31.81732	3305.882	0	
WIRELESS_3888000	4629.527	3883802	431.9999	1296.791	31.18805	3301.549	0	
WIRELESS_3888000	4584.456	3883848	431.9999	1240.5	31.63374	3312.322	0	
WIRELESS_3888000	4622.644	3883809	431.9999	1287.997	31.49771	3303.149	0	
WIRELESS_3888000	4610.446	3883822	431.9999	1273.048	31.26878	3306.13	0	
WIRELESS_3888000	4589.555	3883842	431.9999	1246.906	31.53642	3311.112	0	
WIRELESS_3888000	4613.067	3883819	431.9999	1276.588	30.91489	3305.564	0	
WIRELESS_3888000	4601.086	3883831	431.9999	1261.273	31.46453	3308.348	0	
WIRELESS_3888000	4594.491	3883838	431.9999	1252.98	31.59945	3309.912	0	
WIRELESS_3888000	4588.88	3883843	431.9999	1245.795	31.87372	3311.212	0	
WIRELESS_3888000	4603.648	3883828	431.9999	1264.222	31.74765	3307.678	0	
WIRELESS_3888000	4608.628	3883823	431.9999	1270.429	31.71336	3306.485	0	
WIRELESS_3888000	4597.654	3883834	431.9999	1256.923	31.57623	3309.154	0	
WIRELESS_3888000	4597.839	3883834	431.9999	1256.968	31.80405	3309.067	0	
WIRELESS_3888000	4585.021	3883847	431.9999	1241.266	31.55411	3312.201	0	
WIRELESS_3888000	4583.065	3883849	431.9999	1239.016	31.33735	3312.712	0	
WIRELESS_3888000	4594.96	3883837	431.9999	1254.045	31.00668	3309.909	0	
WIRELESS_3888000	4612.288	3883820	431.9999	1275.324	31.28095	3305.684	0	
WIRELESS_3888000	4608.16	3883824	431.9999	1269.96	31.57623	3306.623	0	
WIRELESS_3888000	4569.789	3883862	431.9999	1222.021	31.97657	3315.792	0	
WIRELESS_3888000	4618.115	3883814	431.9999	1282.164	31.75981	3304.191	0	

Table 6.7.1: AODV Energy Harvesting on, QoS BE(Low)

No	Consumed(mJ)	Remaining (mJ)	Energy Efficiency (%)
1	4604.674	3,883,365	66.8816%
2	4611.884	3,883,388	66.8814%
3	4560.664	3,883,406	66.8820%

4	4562.485	3,883,408	66.8820%
5	4615.861	3,883,384	66.8814%
6	4606.165	3,883,361	66.8816%
7	4560.566	3,883,406	66.8820%
8	4576.683	3,883,420	66.8823%
6	4560.100	3,883,406	66.8820%
10	4587.454	3,883,413	66.8821%
11	4605.348	3,883,365	66.8816%
12	4581.651	3,883,418	66.8822%
13	4606.344	3,883,361	66.8816%
14	4576.205	3,883,424	66.8824%
15	4611.276	3,883,386	66.8814%
16	4611.053	3,883,386	66.8814%
17	4626.527	3,883,370	66.8806%(Lowest)
18	4584.456	3,883,416	66.8821%
19	4622.644	3,883,377	66.8810%
20	4610.446	3,883,360	66.8816%
21	4586.555	3,883,410	66.8820%
22	4613.067	3,883,387	66.8815%
23	4601.086	3,883,366	66.8817%
24	4564.461	3,883,406	66.8816%
25	4588.880	3,883,411	66.8821%
26	4603.648	3,883,366	66.8816%
27	4608.628	3,883,361	66.8816%
28	4567.654	3,883,402	66.8816%

29	4567.836	3,883,402	66.8816%
30	4585.021	3,883,415	66.8821%
31	4583.065	3,883,417	66.8822%
32	4564.660	3,883,405	66.8816%
33	4612.288	3,883,388	66.8814%
34	4608.160	3,883,362	66.8816%
35	4566.786	3,883,430	66.8825% (Highest)
36	4618.115	3,883,382	66.8813%

Table 6.7.2:AODV Energy Efficiency

Energy Efficiency = Remaining Energy/Initial

energy First Sensor

Initial Energy = 3,888,000 mJ, Remaining Energy = 3,883,827 mJ Energy

Efficiency ~99.89%

Energy Efficiency for All Sensors

Have same Initial Energy (3,888,000 mJ)

Consumed energy $\approx$ 4580–4630 mJ

Remaining energy $\approx$ 3,883,800 Mj

The energy efficiency for all sensors

lies approximately

between 99.88% to 99.90%

Battery model								
Device Name	Initial energy	Consumed energy	Remaining energy	Harvested energy	Transmitted energy	Received energy	Idle energy	Sleep energy(mJ)
WIRELESS_3888000	5107.352	3883325	431.9999	1402.525	629.4598	3075.368	0	
WIRELESS_3888000	5095.712	3883336	431.9999	1389.409	627.8286	3078.474	0	
WIRELESS_3888000	5093.211	3883339	431.9999	1386.519	627.5676	3079.125	0	
WIRELESS_3888000	5089.741	3883342	431.9999	1381.469	628.4811	3079.792	0	
WIRELESS_3888000	5098.965	3883333	431.9999	1392.329	629.1988	3077.437	0	
WIRELESS_3888000	5082.2	3883350	431.9999	1371.739	628.9378	3081.523	0	
WIRELESS_3888000	5145.409	3883287	431.9999	1440.397	640.9437	3064.069	0	
WIRELESS_3888000	5142.142	3883290	431.9999	1435.174	642.3792	3064.589	0	
WIRELESS_3888000	5096.636	3883335	431.9999	1379.45	641.4657	3075.72	0	
WIRELESS_3888000	5161.974	3883270	431.9999	1463.449	637.877	3060.648	0	
WIRELESS_3888000	5143.07	3883289	431.9999	1441.267	636.311	3065.492	0	
WIRELESS_3888000	5123.896	3883308	431.9999	1412.48	642.4444	3068.972	0	
WIRELESS_3888000	5128.45	3883304	431.9999	1416.802	644.0756	3067.572	0	
WIRELESS_3888000	5150.618	3883281	431.9999	1447.551	640.0954	3062.971	0	
WIRELESS_3888000	5127.167	3883305	431.9999	1415.635	643.5536	3067.978	0	
WIRELESS_3888000	5131.573	3883300	431.9999	1424.715	639.1167	3067.741	0	
WIRELESS_3888000	5148.139	3883284	431.9999	1445.856	638.399	3063.883	0	
WIRELESS_3888000	5134.979	3883297	431.9999	1424.585	644.4671	3065.927	0	
WIRELESS_3888000	5164.479	3883268	431.9999	1464.326	640.6174	3059.536	0	
WIRELESS_3888000	5130.957	3883301	431.9999	1421.561	642.0529	3067.344	0	
WIRELESS_3888000	5140.143	3883292	431.9999	1433.862	640.9437	3065.337	0	
WIRELESS_3888000	5149.289	3883283	431.9999	1444.68	641.5962	3063.013	0	
WIRELESS_3888000	5127.714	3883304	431.9999	1417.324	642.3139	3068.076	0	
WIRELESS_3888000	5132.643	3883299	431.9999	1421.846	644.2714	3066.526	0	
WIRELESS_3888000	5121.932	3883310	431.9999	1409.352	643.2927	3069.288	0	
WIRELESS_3888000	5203.208	3883229	431.9999	1494.584	662.4759	3046.148	0	
WIRELESS_3888000	5210.293	3883222	431.9999	1504.385	661.2362	3044.672	0	
WIRELESS_3888000	5189.701	3883242	431.9999	1479.417	660.5185	3049.765	0	
WIRELESS_3888000	5225.079	3883207	431.9999	1522.148	661.9539	3040.977	0	
WIRELESS_3888000	5185.148	3883247	431.9999	1473.237	661.171	3050.741	0	
WIRELESS_3888000	5195.696	3883236	431.9999	1484.625	663.2589	3047.812	0	
WIRELESS_3888000	5172.06	3883260	431.9999	1453.701	665.2164	3053.142	0	
WIRELESS_3888000	5198.903	3883233	431.9999	1489.72	661.8887	3047.294	0	
WIRELESS_3888000	5200.168	3883232	431.9999	1489.43	664.1724	3046.565	0	
WIRELESS_3888000	5180.007	3883252	431.9999	1463.934	664.7597	3051.313	0	
WIRELESS_3888000	5166.763	3883265	431.9999	1452.442	658.6915	3055.63	0	

Table 6.7.3: RLP Energy Harvesting on, QoS BE(Low)

First Sensor

Initial Energy = 3,888,000 mJ

Consumed Energy = 5107.352 mJ Energy Efficiency ~99.87%

Energy Efficiency Range for All Sensors consumed energy varies between:

Minimum $\approx$ 5082 mJ

Maximum $\approx$ 5225 mJ

Highest Efficiency (Lowest Consumption ~5082 mJ)

Efficiency $\approx$ 99.87%

Lowest Efficiency (Highest Consumption ~5225 mJ)

Efficiency $\approx$ 99.86%

Sensor	Consumed Energy(mJ)	Remaining Energy(mJ)	Energy Efficiency (%)
1	5107.352	3,882,893	99.868%

2	5095.712	3,882,904	99.869%
3	5093.211	3,882,907	99.869%
4	5089.741	3,882,910	99.869% (Highest)
5	5098.965	3,882,901	99.868%
6	5082.200	3,882,918	99.870% (HighestOverall)
7	5145.409	3,882,855	99.867%
8	5142.142	3,882,858	99.867%
9	5096.636	3,882,903	99.868%
10	5161.974	3,882,838	99.867%
11	5143.070	3,882,857	99.867%
12	5123.896	3,882,876	99.868%
13	5128.450	3,882,872	99.868%
14	5150.618	3,882,849	99.867%
15	5127.167	3,882,873	99.868%
16	5131.573	3,882,868	99.867%
17	5148.139	3,882,852	99.867%
18	5134.979	3,882,865	99.867%
19	5164.479	3,882,836	99.867%
20	5130.957	3,882,869	99.868%
21	5140.143	3,882,860	99.867%
22	5149.289	3,882,851	99.867%
23	5127.714	3,882,872	99.868%
24	5132.643	3,882,867	99.867%
25	5121.932	3,882,878	99.868%

26	5203.208	3,882,797	99.866%
27	5210.293	3,882,790	99.866% (Lowest)
28	5189.701	3,882,810	99.866%
29	5225.079	3,882,775	99.865% (LowestOverall)
30	5185.148	3,882,815	99.866%
31	5195.696	3,882,804	99.866%
32	5172.060	3,882,828	99.867%
33	5198.903	3,882,801	99.866%
34	5200.168	3,882,800	99.866%
35	5180.007	3,882,820	99.867%
36	5166.763	3,882,833	99.867%

Table 6.7.4: RLP Energy Efficiency

6.8 Packet Delivery Ratio (PDR)

Packet Delivery Ratio (PDR) is defined as the ratio of the number of packets successfully received to the total number of packets transmitted. The simulation results show that the PDR varies across different sensor nodes, with values ranging approximately between 0.14% and 0.17%. The variation in PDR is due to factors such as packet loss, collisions, and network congestion. Higher PDR values indicate better reliability of data transmission. The results suggest that network performance can be improved by optimizing scheduling and reducing contention in the wireless medium.

Packet Delivery Ratio (PDR) is defined as:

$$PDR = \frac{\text{Total Packets Received}}{\text{Total Packets Generated}} \times 100$$

Packets Generated = 225000 (for each sensor)

Packets Received ≈ 325 to 389 (varies per app)

Highest PDR Sensor

- App17_CBR → 389 packets received

$$PDR = \frac{389}{225000} \times 100 \approx 0.173\%$$

Lowest PDR Sensor

- App18_CBR → 325 packets received

$$PDR = \frac{325}{225000} \times 100 \approx 0.144\%$$

App	Packets Received	PDR (%)
App1	358	0.156%
App2	356	0.160%
App3	335	0.146%
App4	353	0.157%
App5	355	0.158%
App6	338	0.150%
App7	328	0.146%
App8	348	0.155%
App6	376	0.167%
App10	342	0.152%
App11	371	0.165%
App12	343	0.152%
App13	340	0.151%
App14	326	0.146%
App15	362	0.161%
App16	368	0.164%

App17	386	0.173% (Highest)
App18	325	0.144% (Lowest)
App16	353	0.157%
App20	344	0.153%
App21	335	0.146%
App22	366	0.163%
App23	334	0.148%
App24	374	0.166%
App25	357	0.156%
App26	333	0.148%
App27	350	0.156%
App28	382	0.170%
App26	336	0.146%
App30	367	0.163%
App31	334	0.148%
App32	362	0.161%
App33	335	0.146%
App34	362	0.161%
App35	326	0.145%
App36	386	0.171%

Table 6.8.1: PDR Table of AOD

Packet Delivery Ratio (PDR) is defined as:

$$PDR = \frac{\text{Total Packets Received}}{\text{Total Packets Generated}} \times 100$$

Packets Generated = 225000 (for each sensor)

App	Packets Received	PDR (%)
App1	358	0.156%
App2	356	0.160%
App3	335	0.146%
App4	353	0.157%
App5	355	0.158%
App6	338	0.150%
App7	328	0.146%
App8	348	0.155%
App6	376	0.167%
App10	342	0.152%
App11	371	0.165%
App12	343	0.152%
App13	340	0.151%
App14	326	0.146%
App15	362	0.161%
App16	368	0.164%

App17	386	0.173% (Highest)
App18	325	0.144% (Lowest)
App16	353	0.157%
App20	344	0.153%
App21	335	0.146%
App22	366	0.163%
App23	334	0.148%
App24	374	0.166%
App25	357	0.156%
App26	333	0.148%
App27	350	0.156%
App28	382	0.170%
App26	336	0.146%
App30	367	0.163%
App31	334	0.148%
App32	362	0.161%
App33	335	0.146%
App34	362	0.161%
App35	326	0.145%
App36	386	0.171%

Table 6.8.2: PDR Table of RLP

6.9 Impact of Edge-Based TRAPS Mechanism

The implementation of the TRAPS mechanism at the gateway significantly improves network performance by reducing redundant task execution and minimizing unnecessary data transmission. By processing tasks locally at the gateway, the system reduces network load and improves response time. The energy-aware scheduling ensures balanced utilization of sensor nodes, thereby extending network lifetime. The results confirm that integrating edge computing with intelligent task scheduling enhances overall system efficiency and makes the network more suitable for real-time IoT applications.

6.10 Performance Comparison of AODV and RPL

The table compares AODV and RPL based on PDR, cloud delay, and energy efficiency. Both protocols show similar PDR values, with the same highest and lowest applications. RPL reaches the cloud slightly faster in some cases, while both have nearly equal delay at the end. Energy efficiency is very high for both (above 99.8%), with only minor differences between sensors. Overall, both protocols perform almost equally with small variations.

Metrics	AODV	RPL
Highest PDR	App17_CBR(0.173%)	App28_CBR(0.166%)
Lowest PDR	App18_CBR(0.144%)	App6_CBR(0.130%)
Reached Cloud First	App9_CBR(131E+08 μ s)	App17_CBR(126E+08 μ s)
Reached Cloud Last	App35_CBR(149E+08 μ s)	App16_CBR(149E+08 μ s)
Highest Energy Efficiency	Sensor35(~99.8825%)	Sensor6(~99.870%)
Lowest Energy Efficiency	Sensor17(~99.8809%)	Sensor29(~99.865%)

Table 6.10.1: Performance Comparison of AODV and RPL

6.11 Hardware Results and Analysis

6.11.1 Hardware Setup Output

The hardware setup consists of Arduino, Raspberry Pi, and multiple sensors including DHT11 and MQ-3. The system successfully captures real-time environmental data and establishes communication between components. Proper wiring and power supply ensure stable operation of the system.

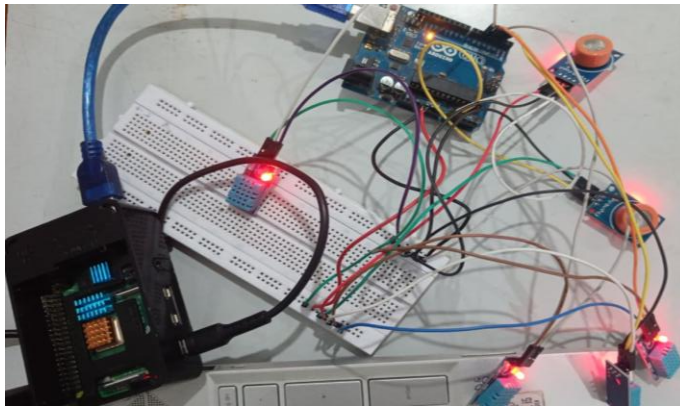


Figure 6.11.1: Hardware Setup Output

6.11.2 Sensor Data Output (Arduino / Raspberry Pi)

The Arduino collects sensor data and transmits it via serial communication to the Raspberry Pi. The output displays real-time values such as temperature, humidity, and gas levels. This confirms accurate data acquisition and processing at the device level

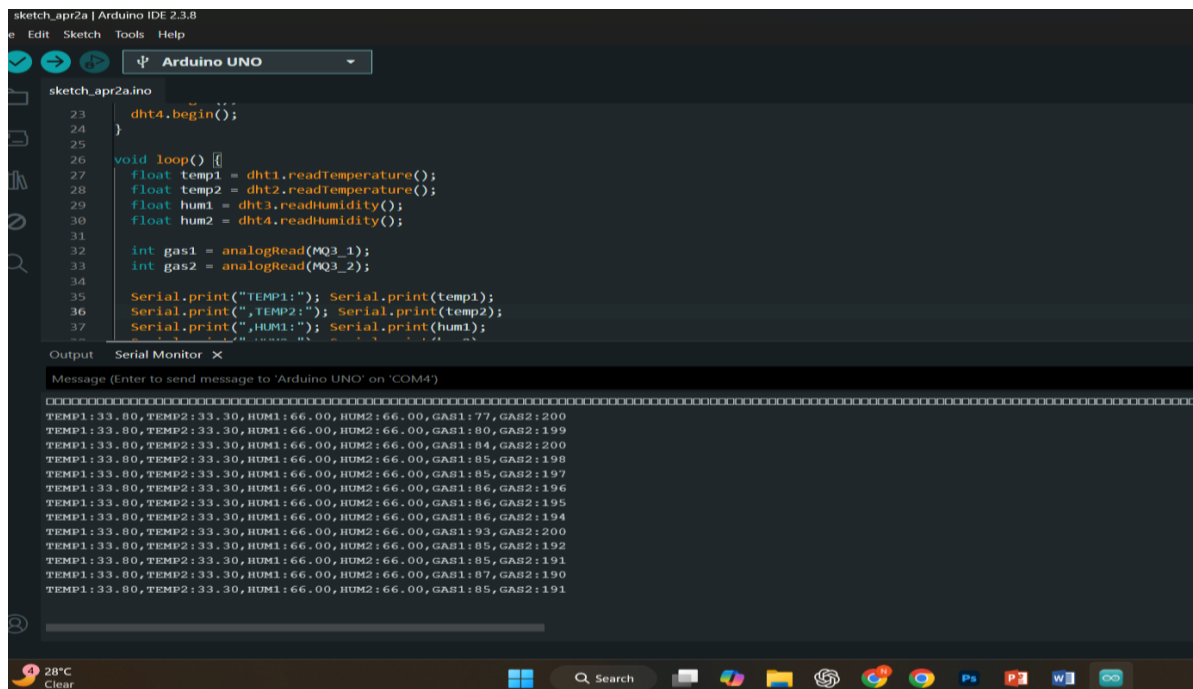


Figure 6.11.2(a): Sensor Data Output (Arduino)

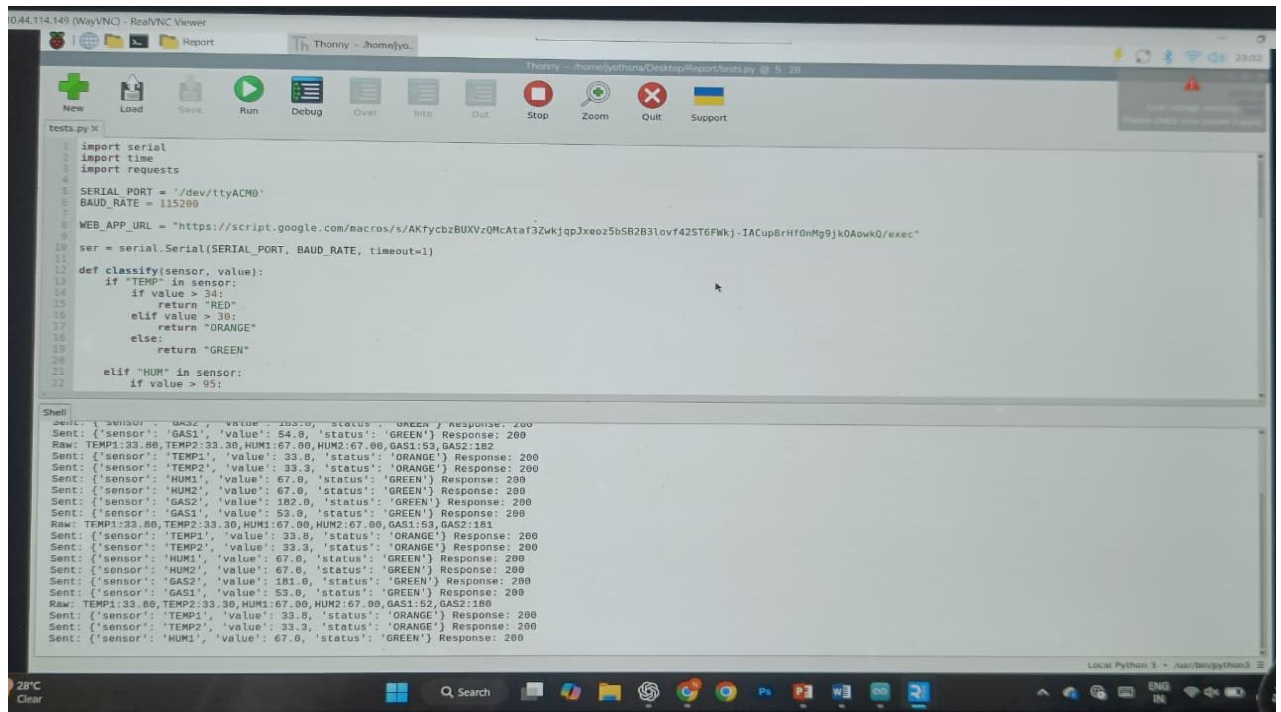


Figure 6.11.2(a): Sensor Data Output (Raspberrypi)

6.11.3 Data Transmission to Gateway

The Raspberry Pi acts as a gateway, receiving sensor data from Arduino and processing it efficiently. The data is formatted and prepared for transmission to the cloud. This ensures reliable and continuous data flow within the system.

6.11.4 Cloud Storage Results (Google Sheets)

- The processed sensor data is uploaded to Google Sheets for cloud storage and monitoring. The data is updated in real-time, enabling remote access and analysis.
- This demonstrates successful integration of IoT with cloud services.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
3	GAS2	179	GREEN												
4	HUM2	66	GREEN												
5	HUM1	67	GREEN												
6	TEMP2	33.3	ORANGE												
7	TEMP1	33.8	ORANGE												
8	GAS1	49	GREEN												
9	GAS2	177	GREEN												
10	HUM2	66	GREEN												
11	HUM1	67	GREEN												
12	TEMP2	33.3	ORANGE												
13	TEMP1	33.8	ORANGE												
14	GAS1	50	GREEN												
15	GAS2	178	GREEN												
16	HUM2	66	GREEN												
17	HUM1	67	GREEN												
18	TEMP2	33.3	ORANGE												
19	TEMP1	33.8	ORANGE												
20	GAS1	49	GREEN												
21	GAS2	178	GREEN												
22	HUM2	66	GREEN												
23	HUM1	67	GREEN												
24	TEMP2	33.3	ORANGE												
25	TEMP1	33.8	ORANGE												
26	GAS1	49	GREEN												

Figure 6.11.4: Cloud Storage Results (Google Sheets)

6.11.5 Web Application Output

The web application displays live sensor data with status indicators for easy monitoring. Different parameters are visually represented using color-coded outputs. This provides a user-friendly interface for real-time system observation.

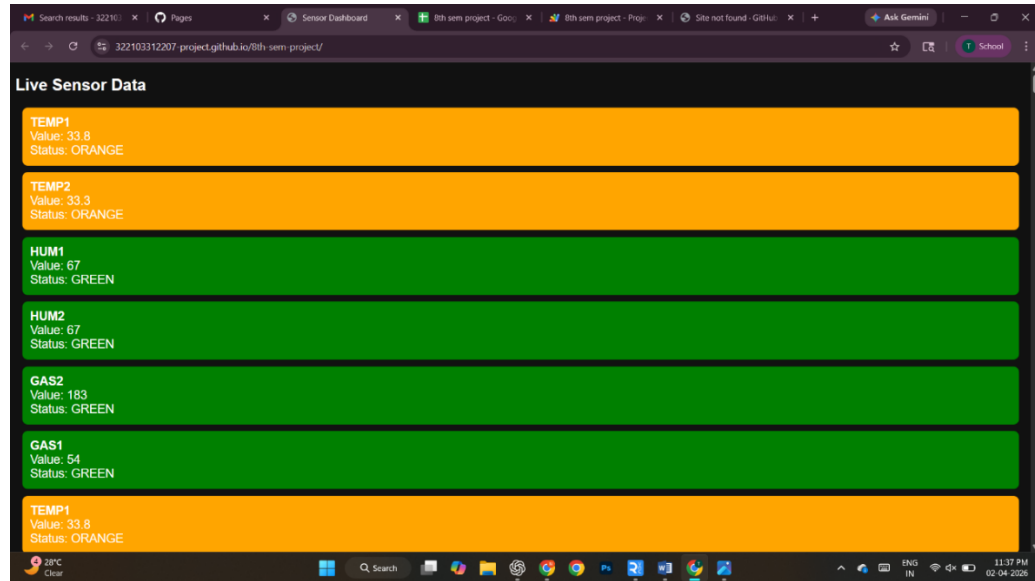


Figure 6.11.5: Web Application Output

6.12 Summary

This chapter presented a detailed analysis of the simulation results obtained from different scenarios. The study highlights the limitations of traditional scheduling techniques and the importance of transmission control in wireless networks. The results demonstrate that edge-based processing using the TRAPS mechanism improves network performance by reducing latency, optimizing energy consumption, and enhancing data transmission efficiency.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project presents the design and analysis of an IoT-based Wireless Sensor Network integrated with an edge computing gateway to improve performance. It addresses issues in traditional cloud-based IoT systems such as high latency, bandwidth usage, and energy consumption.

An intelligent gateway using the TRAPS mechanism enables local data processing, reducing redundant tasks and unnecessary cloud communication. The system was evaluated using NetSim under various scenarios, showing that traditional scheduling methods like FIFO are insufficient in wireless environments. AODV offered slightly better energy efficiency, while RPL proved more suitable for IoT applications.

Performance metrics such as throughput, delay, energy consumption, and packet delivery ratio confirmed improved efficiency and reliability. The system was also implemented using sensors, Arduino, and Raspberry Pi, with data stored in Google Sheets and monitored via a web application.

Overall, the project demonstrates that integrating edge computing with intelligent scheduling enhances IoT network performance and scalability for real-time applications.

7.2 Future Scope

The proposed system can be improved by integrating advanced technologies for real-world applications. Future enhancements include adding machine learning at the edge gateway for predictive analysis and adaptive decision-making.

Advanced routing protocols and hybrid scheduling can further improve efficiency and reliability, while security features like encryption and authentication can ensure safe data transmission. The system can also be scaled for large applications such as smart cities, healthcare, and industrial automation.

Additionally, upgrading hardware with advanced IoT devices and edge platforms can enhance accuracy, scalability, and overall performance.

7.3 Final Remarks

The project provides a comprehensive approach to improving IoT network performance by combining edge computing with intelligent task scheduling. The results highlight that efficient data processing at the gateway level is a key factor in achieving low latency, reduced energy consumption, and enhanced network scalability. Furthermore, the hardware implementation using sensors, Arduino, and Raspberry Pi validates the practical feasibility of the system by demonstrating real-time data acquisition, processing, and cloud-based monitoring in a real-world environment.

REFERENCES

- [1] S. Kumar et al., “Enhancing Network Efficiency in IoT Sensor Systems Through Edge-Based Task Processing,” *Project Report*, Department of Electronics and Communication Engineering, GVPCE (A).
- [2] “Designing Gateway through Edge Computing (TRAPS) – The Ultimate Idea of Communication,” *Project Presentation*, GVPCE (A).
- [3] “Performance Comparison of AODV and RPL in IoT-Based Wireless Sensor Networks,” *Project Report*, NetSim Simulation Study.
- [4] T. Clausen and P. Jacquet, “Optimized Link State Routing Protocol (OLSR),” IETF RFC 3626, 2003.
- [5] C. Perkins, E. Belding-Royer, and S. Das, “Ad hoc On-Demand Distance Vector (AODV) Routing,” IETF RFC 3561, 2003.
- [6] T. Winter et al., “RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks,” IETF RFC 6550, 2012.
- [7] NetSim Network Simulator, Tetcos, “NetSim User Manual and Documentation,” Version 14.4.
- [8] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [9] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog Computing and Its Role in the Internet of Things,” *Proceedings of the MCC Workshop on Mobile Cloud Computing*, 2012.
- [10] K. Ashton, “That ‘Internet of Things’ Thing,” *RFID Journal*, 2009.

